COMMANDER

CALC

SPREADSHEET FOR COMMANDER X16

Reference Manual

Release 1.0 • For the Commander X16

A native spreadsheet for the Commander X16. Sixteen worksheets, sixteen thousand cells to a sheet, fourteen functions and a formula compiler, in a thirty-kilobyte program running on a 65C02 at eight megahertz.

Commander Calc Reference Manual
Release 1.0

This manual describes Commander Calc as it is built from the source in `x16sheet/`. Where the manual and the program disagree, the program is right and the manual is wrong; please say so.

The program is distributed as seventeen files: `CMDRCALC.PRG` and `OVL1.BIN` through `OVL16.BIN`. All seventeen must be present on the same device and in the same directory. Appendix E lists them.

Commander Calc runs on a Commander X16 with 512K of banked RAM and a CMDR-DOS storage device at unit 8, and on the `x16emu` emulator with an equivalent configuration. Chapter 2 gives the details.

Commander X16 is the name of the computer this program was written for. *Lotus*, *1-2-3* and *Excel* are named in this manual only where the behaviour of Commander Calc is being compared with theirs; they are the trademarks of their respective owners and this program is not connected with, endorsed by or derived from any of them.

Set in Nimbus Roman, Nimbus Sans and Nimbus Mono. Every illustration in this manual, including the logo above, is drawn in TeX — there is not one imported picture in the book.

How to Use This Manual

This manual is in two parts and a set of appendices.

Part One, *Using Commander Calc*, is written to be read. It starts at the screen and works outwards: what is on the display, how the keyboard moves the cursor, how a value gets into a cell, what the menus contain, how files are opened and saved, and what the commands that act on more than one cell will and will not do. Read it once, in order, and you will not need to read it again.

Part Two, *Reference*, is written to be looked things up in. Every menu command, every key, every function and every message the program can display has an entry, and each entry stands on its own. Nothing in Part Two assumes you have read Part One, and nothing in it is a tutorial.

The appendices are the numbers: how much the program can hold, what a workbook file contains byte for byte, how a number is represented, and where the machine's memory goes. You do not need any of it to use Commander Calc. You need all of it to know what the program will do at the edges.

What this manual assumes

That you have used a spreadsheet before — that a cell, a row, a column and a formula are already familiar. Commander Calc is small, and this manual is correspondingly short on explanations of what a spreadsheet is for.

It does *not* assume that the spreadsheet you have used before was this one. Commander Calc is not a clone of anything. It leaves out a great deal that other programs have, and a few of the things it does keep, it does differently. Where that is true, this manual says so plainly rather than leaving you to find out. Appendix F collects every such difference in one list.

Conventions

Convention	Meaning
Convention	**Meaning**
F7	A key on the keyboard. Press it.
CTRL + C	Two keys pressed together: hold the first, press the second.
File > Import	A menu path. Open the **File** menu, then choose **Import**.
`=SUM(A1:A10)`	Something you type, or something the program displays, character for character.
`no matching files on the card`	A message the program puts on the screen. Messages are quoted exactly, including their punctuation and their spelling.
`#DIV/0!`	An error value, as it appears in a cell.
`CMDRCALC.PRG`	A file name.
`A1`, `B2:D40`	A cell and a range of cells.

Two kinds of aside interrupt the text.

NOTE A note tells you something useful that is not obvious, usually because it is a consequence of how the program is built rather than of how it is used.

CAUTION A caution tells you about something that costs you work: a command that cannot be undone, a limit that silently truncates, or a file that will not come back. Cautions are rare, and every one of them is there because the alternative is losing a workbook.

Keys and the Commander X16 keyboard

Commander Calc puts the machine into ISO mode at startup, which is what makes the keyboard produce ordinary ASCII: unshifted letters arrive lowercase, shifted letters uppercase. Everything in this manual that talks about typing assumes that.

Several keys on the Commander X16 keyboard send the same code as a control combination, and the program cannot tell them apart. CTRL + M is RETURN; CTRL + I is TAB. The full list is in Chapter 16. Where this manual names a key, it names the one you are most likely to be pressing.

F12 is not used by Commander Calc and never will be: on the emulator it opens the machine's own debugger, and a program that took the key would take that away.

The page numbers

Pages are numbered by chapter. Page `9-4` is the fourth page of Chapter 9; page `B-2` is the second page of Appendix B. The black tab on the outer edge of each page steps one notch down the page per chapter, so the manual can be opened at a chapter by fanning it rather than by reading the numbers.

Contents

List of Figures

PART ONE

Using Commander Calc

How the program works and how to work it: the screen, the keyboard, entering and editing data, the menus, files, worksheets and the commands that operate on a whole block of rows at once. Read this part through once and you will not need to read it again.

CHAPTER 1

Introduction

Commander Calc is a spreadsheet program for the Commander X16. It holds up to sixteen worksheets in one workbook, sixteen thousand cells on each of them, and recalculates a formula written in terms of any of the others. It reads and writes its own file format, imports and exports comma-separated text, and imports and exports Excel `.XLSX` workbooks. It does all of that in a thirty-kilobyte program on a 65C02 running at eight megahertz.

That last sentence is the reason for most of this manual. Commander Calc is not a scaled-down version of a large program; it is a small program, designed as one, and the shape of it shows. Knowing what it does *not* do is as useful as knowing what it does, so this manual says both.

What Commander Calc is

- **A worksheet program.** A grid of 256 columns by 65,535 rows, addressed `A1` to `IV65535`. Cells hold numbers, text, dates, `TRUE` and `FALSE`, formulas, and error values.
- **A workbook program.** Up to sixteen worksheets in one file, with their own names and their own tabs, and formulas that reach across them: `=SUM(Q1!B2:B40)` does what it looks like.
- **A block editor.** Mark a rectangle of any size, then copy it, paste it somewhere else, or clear it in one command.
- **Formulas that move with their cells.** A pasted formula has its references rewritten by the distance the block travelled, and so does one carried along by a sort or by an inserted row. A `$` pins whatever should not move.
- **Charts.** Three commands draw the numbers in the cursor's column as bars, as a line, or as a pie, labelled from the column beside them — and any of them can be saved to the card as a picture file.
- **A calculator with fourteen functions.** **SUM**, **AVERAGE**, **MIN**, **MAX**, **COUNT**, **IF**, **AND**, **OR**, **NOT**, **ABS**, **ROUND**, **INT**, **MOD** and **LEN**. That is the whole list; Chapter 18 documents every one.

- **A program that reads other people's files.** An Excel workbook saved as `.XLSX` can be opened, with its numbers, text, dates, number formats, column widths, multiple worksheets and — as far as the fourteen functions reach — its formulas.
- **A program that writes files other people can read.** A workbook can be written back out as `.XLSX` or as `.CSV`.

What Commander Calc is not

It is not Lotus 1-2-3 and it is not Excel, and it does not pretend to be. The list below is the short version; Appendix F is the long one.

- **There is no cut or move.** Copy and paste, and delete what you left behind.
- **There are no colour or font commands.** Commander Calc sets column widths and freezes headings, and it displays eight number formats, bold and three alignments faithfully — but the only way to get a *format* into a workbook is to import a file that already has one. Nothing on the menus sets a number format, bold, or a colour.
- **There are no text functions, lookup functions or date functions.** No **LEFT**, no **VLOOKUP**, no **SUMIF**, no **TODAY**.
- **Undo is one level deep.** One cell. Sorting has its own separate one-level undo, and a pasted block cannot be taken back at all.
- **A formula longer than 128 characters is never rewritten.** It is left exactly as it was rather than half-adjusted, wherever it is moved to.
- **There are no macros, no printing and no windows**, and a chart covers one column at a time.

NOTE Most of these are absences rather than defects: work that has not been done rather than work that was done wrong. Where something is actually broken — and there are one or two — this manual says so at the point where you would meet it.

What you need

You need	Notes
A Commander X16	Or the `x16emu` emulator, which is what most of this manual was written against.
512K of banked RAM	The machine's standard fitting. The workbook lives entirely in banked RAM, so more of it means a bigger workbook; Commander Calc will use up to the 2M maximum if the machine has it.
A storage device at unit 8	An SD card under CMDR-DOS, or the emulator's host filesystem.

You need	Notes
An 80 by 60 text display	The default. Commander Calc adapts to a smaller text mode and refuses to run in anything shorter than ten lines.
A mouse	Optional. Everything can be done from the keyboard; the mouse is supported for menus, dialogues, moving the cursor and scrolling.

Where to go next

If you want to	Read
Get the program running	Chapter 2
Understand the display	Chapter 3
Type something into a cell	Chapters 4 and 5
Save your work	Chapter 8
Open a spreadsheet from another program	Chapter 9
Look up one command	Chapter 15
Look up one key	Chapter 16
Look up one function	Chapter 18
Draw a picture of some numbers	Chapter 14
Understand a message on the screen	Chapter 21
Know exactly how big a workbook can be	Appendix A
Know what is missing and why	Appendix F

CHAPTER 2

Installing and Starting

The files

Commander Calc is seventeen files, and **all seventeen must be in the same directory of the same device**.

File	What it is
CMDRCALC.PRG	The program. This is the one you load.
OVL1.BIN… OVL16.BIN	The rest of the program. Commander Calc is larger than the machine's memory, so it keeps most of itself on the card and reads in whichever part it needs.

Together they come to about 135K. The names must be spelled exactly as they are above, in capitals: the program builds them itself when it goes looking, and it will not find a file named any other way.

CAUTION Nothing is checked at startup. Commander Calc starts perfectly happily with files missing and then fails, silently, at the first command that needs one of them: a menu that will not open, a file command that returns immediately having done nothing, a formula that will not go into a cell. If the program is behaving as though the keyboard is disconnected, count the files — there should be seventeen.

Installing on an SD card

Copy the seventeen files into a directory of a CMDR-DOS-formatted card, put the card in the machine, and:

Procedure

1. Type `LOAD"CMDRCALC.PRG",8` and press RETURN.
2. Type `RUN` and press RETURN.

Commander Calc looks for the rest of itself on whichever device it was loaded from, falling back to unit 8 if that cannot be determined — which is almost always the right answer anyway.

Your workbooks can live in the same directory. They do not have to; but the file dialogue lists only the current directory and has no way to change it, so in practice they will.

Running under the emulator

The `x16emu` emulator serves a host directory to the program as though it were unit 8, which is the most convenient way to work.

```
x16emu -rom rom.bin -fsroot . -prg CMDRCALC.PRG -run
```

`-fsroot` names the directory holding the seventeen files and your workbooks. A `rom.bin` is required: all of Commander Calc's arithmetic is done by the floating-point library in bank 4 of the Commander X16 ROM, and without a ROM there is nothing to call.

Two flags change what you are testing.

Flag	Effect
`-ram 512`	The memory a real machine ships with. This is what to use if you want to know whether a workbook will fit on stock hardware.
`-ram 2048`	The emulator's maximum, and the machine's. The difference between the two is the difference between a workbook of a few thousand cells and one of a few hundred thousand.
`-mhz 8`	The speed of a real Commander X16. Anything faster is a convenience for development and will make imports look quicker than they are.

NOTE The `-echo` flag will not show you Commander Calc's display. The program draws directly into video memory rather than going through the KERNAL's character output, so there is nothing for `-echo` to intercept. To capture the screen, record with `-gif` and take a frame from the recording.

Starting

Commander Calc starts with one empty worksheet named `Sheet1`, the cursor on `A1`, no file name, and the status line reading `READY` with the amount of free banked memory beside it.

```
 READY  A1          516088 bytes free
```

Figure 2-1 The status line at startup on a 512K machine

The figure that appears there is the whole of banked RAM, measured once. On a 2M machine it reads something over two million. It is a statement of how large a workbook this machine can hold, and it does not change as you fill the workbook up.

Quitting

Press F9, or choose **File > Quit**.

If the workbook has been modified — if the status line says `MODIF` — Commander Calc asks first:

```
Quit

discard unsaved changes?
Y = yes, any other key = no
[  Yes  ]   [  No   ]
```

Figure 2-2 The confirmation shown when quitting with unsaved changes

Y quits and throws the work away. *Any* other key cancels — there is no separate N; anything that is not Y means no. You can also click the buttons.

On exit the screen is cleared to white on blue and you are back in BASIC.

If something is wrong

Symptom	Cause
Menus will not open; file commands do nothing	One of the program's files is missing or mis-named. Check that all sixteen `OVL`*n*`.BIN` files are present beside `CMDRCALC.PRG`, spelled in capitals.
`no matching files on the card` when opening a workbook you know is there	The dialogue lists only the current directory, only files ending `.X16S`, and only the first thirty-two of them. See Chapter 8.

Symptom	Cause
The status line says `STACK!`	An internal limit has been exceeded. Save your work under a new name and restart. See Chapter 21.
Arithmetic gives nonsense, or the machine drops to BASIC	There is no ROM, or the wrong one. The floating-point library lives in ROM bank 4.
Nothing is legible	The machine is in a text mode Commander Calc did not expect. It wants 80 by 60.

CHAPTER 3

The Commander Calc Screen

Commander Calc uses the whole of an 80-column by 60-line text screen and draws every character of it itself, straight into video memory. Nothing scrolls off the top, nothing is printed, and there is no command line: the display is a single fixed layout, and every one of its sixty lines has a job.

Figure 3-1 is the whole of it.

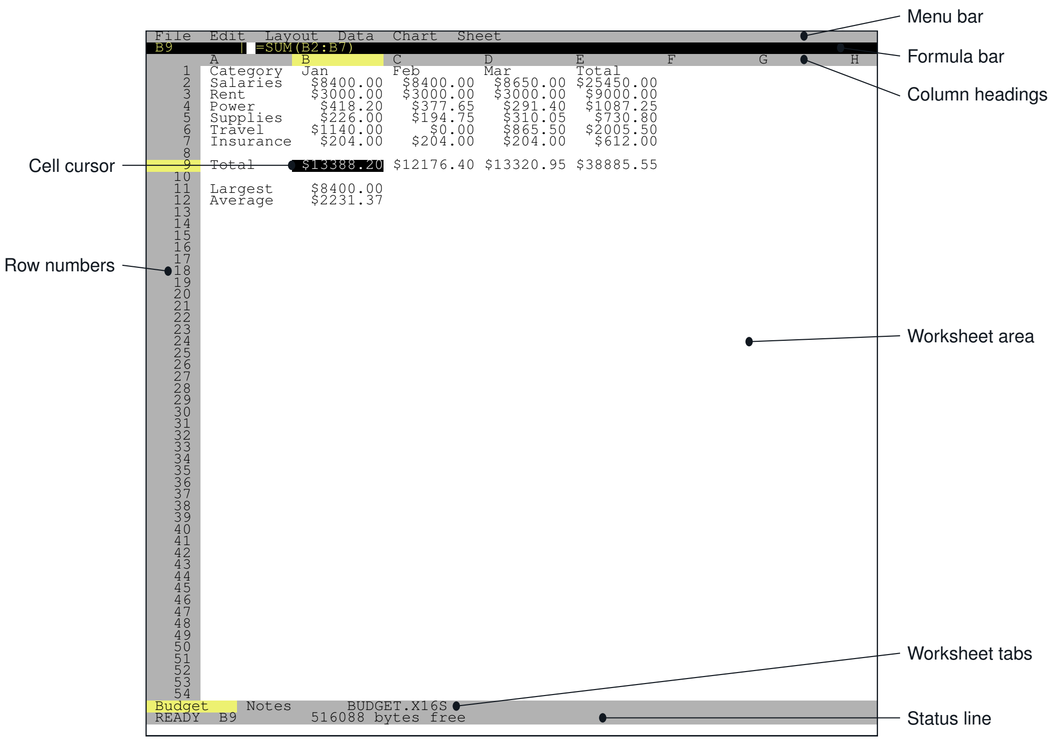


Figure 3-1 The Commander Calc screen

What is on each line

The layout is computed from the screen size at startup, so the line numbers below are for the 80 by 60 display Commander Calc asks for and normally gets. If the machine is in a smaller text mode the program adapts, and it refuses to run at all in anything shorter than ten lines.

Line	Contents
0	The menu bar, whether or not a menu is open. It always reads: `File   Edit   Layout   Data   Chart   Sheet`
1	The formula bar: the address of the current cell, a vertical bar, and what that cell really contains.
2	The column headings, `A` through `IV`.
3–56	The worksheet: fifty-four rows of cells with their row numbers down the left.
57	The worksheet tabs, and after them the name of the workbook file.
58	The status line.
59	The message line. Blank almost always; the **Find** prompt and the **Delete row** question are the only things that use it.

The formula bar

The formula bar is the line that tells you the truth about a cell. The worksheet shows you what a cell *displays* — rounded, formatted, and cut off at the column edge. The formula bar shows you what it *contains*.

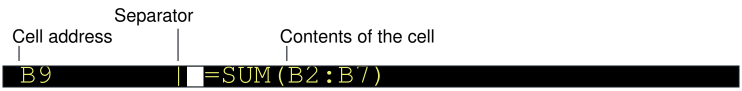


Figure 3-2 The formula bar, showing a formula cell

The address occupies nine characters, which is exactly enough for `IV65535`, the last cell there is. After the separator come sixty-eight characters of cell contents:

- For a formula, the formula as you wrote it — `=SUM(B2:B7)`, not `17388.2`.
- For a number, the number unformatted. A cell displaying `$8.50` in the worksheet shows `8.5` here.
- For text, the text.
- For a blank cell, nothing.

While you are typing, the formula bar becomes the edit line: it shows what you have typed so far, with one character highlighted to mark the insertion point. If what you are typing is longer than sixty-eight characters, the line scrolls horizontally to keep that point in view.

NOTE The formula bar is the only place a formula is ever visible. Commander Calc has no command that shows formulas in the worksheet in place of their results.

The status line

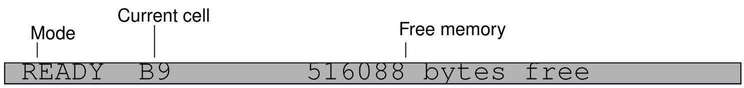


Figure 3-3 The status line

The mode indicator

Five characters at the left of the status line, and there are exactly three things they can say.

Indicator	Meaning
`READY`	Commander Calc is waiting for you. Nothing has been changed since the workbook was last saved or opened.
`MODIF`	The same, except that the workbook has been modified. Leaving the program, opening another workbook or starting a new one will ask you to confirm first.
`EDIT`	You are typing into a cell. The formula bar is showing what you have typed, not what the cell contains.

`MODIF` appears the moment anything is changed and stays until the workbook is saved, opened or replaced. It is the only warning you get that there is work in memory which is not on the card.

The current cell

Eight characters giving the address of the cell the cursor is on — the same address as in the formula bar, repeated at the other end of the screen so that it is near whichever end you happen to be looking at.

The free memory readout

The rest of the status line is a count of free banked RAM, as a plain decimal number followed by `bytes free`. On a machine with the standard 512K you will see about half a million; on a fully expanded 2M machine, about two million.

NOTE This figure is measured once, when the program starts, and is never updated. It tells you how large a workbook the machine can hold. It does not tell you how much of it you have used.

One other thing can appear in this field. If the C stack ever overflows, Commander Calc replaces the whole status text with `STACK!`. See Chapter 21.

The recalculation indicator

The last few characters of the status line are empty almost always. One word can appear there:

Indicator	Meaning
`CALC`	**The answers on screen are stale.** Automatic recalculation has been switched off and something has changed since the last one.

Nothing is shown at any other time — not while automatic recalculation is on, and not while it is off but everything is up to date. The indicator marks the one state worth interrupting you about. Chapter 20 explains the switch.

The worksheet area

Fifty-four rows of cells, with a six-character gutter down the left holding the row numbers, and columns ten characters wide unless a workbook file says otherwise. That gives seven whole columns on screen and a four-character sliver of the eighth.

The cell cursor

The cell the cursor is on is drawn in reverse video — white on black — for the full width of its column. At the same time its row number and its column heading are highlighted in yellow, which is what lets you find the cursor at a glance in a screen full of numbers.

How values are placed in a cell

A value is drawn inside its own column and nowhere else.

- Numbers, and cells holding dates, are set to the right of the column.
- Text, `TRUE` and `FALSE`, and error values are set to the left.
- A formula is aligned by what it produced, not by the fact that it is a formula: a formula returning a number goes right, one returning text goes left.
- A workbook that came from a file may carry an explicit left or right alignment for a cell, which overrides all of the above.

CAUTION A value too wide for its column is cut off at the column edge. It is not spilled into the empty cell next door, and there is no #### marker to tell you it happened — what you see is a shortened number that still looks like a number. If a column of figures looks wrong, check the formula bar before you check the arithmetic. Column widths cannot be changed from the keyboard; see Chapter 19.

The worksheet tabs

Line 57 carries one tab per worksheet, ten characters each, with the active one highlighted in yellow. A worksheet name may be up to thirty-one characters long, but only the first nine of them fit on a tab.

After the last tab, if there is room left on the line, Commander Calc prints the name of the workbook file. A workbook that has never been saved has no name there.

Colours

Commander Calc uses eight of the Commander X16's sixteen colours, and always the same eight for the same purposes.

Area	Colour
Worksheet cells	Black on white
Cell cursor	White on black
Row and column headings	Black on light grey
Heading of the cursor's row and column	Black on yellow
Menu bar, tabs, status line	Black on light grey
Open menu title, highlighted menu item	Black on yellow
Menu items	White on medium grey
Formula bar	Yellow on black
Dialogue box body	Black on light grey
Dialogue title bar, buttons, selected row	White on blue

None of these can be changed.

CHAPTER 4

Moving Around the Worksheet

The worksheet is 256 columns by 65,535 rows and the screen shows seven columns and fifty-four rows of it. This chapter is about getting from one part of it to another.

Addresses

A cell is named by its column letter and its row number, in that order and with nothing between them: `A1`, `B9`, `IV65535`.

Columns run `A` to `Z`, then `AA` to `AZ`, then `BA` onwards, and stop at `IV` — the 256th column.

A	B	C	...	Z	AA	AB	...	AZ	BA	...	IU	IV
1	2	3		26	27	28		52	53		255	256

Column letters, and the column numbers they stand for

Figure 4-1 How the columns are named

Rows are numbered from 1. The last cell in the worksheet is `IV65535`.

Moving one cell at a time

Key	Moves the cursor
↑	Up one row
↓	Down one row
←	Left one column
→	Right one column
RETURN	Down one row — the same as ↓
TAB	Right one column
SHIFT + **TAB**	Left one column

Movement stops at the edges. There is no wrapping from column `IV` to column `A`, and pressing ↑ on row 1 does nothing.

Moving a screen at a time

Key	Moves the cursor
PGUP	Up fifty-four rows — one screenful
PGDN	Down fifty-four rows

The distance is one screenful, whatever a screenful currently is: on a shorter text display these keys move correspondingly less.

Jumping

Key	Goes to
HOME	Column `A` of the row you are on
SHIFT + HOME	Cell `A1`
END	Column `IV` of the row you are on

CAUTION END goes to column `IV` — the last column that exists, not the last column you have used. In an empty worksheet that is 255 columns of nothing. SHIFT + HOME is the way back.

There is no **Go to** command and no way to type an address to jump to.

Scrolling

The screen follows the cursor; you do not scroll the screen independently.

Moving off the bottom or the top of the visible rows scrolls by one row. PGUP and PGDN scroll by a full screen. Moving off the right-hand edge scrolls left by whole columns, one at a time, until the cursor's column fits on the screen completely — so a wide column can push several narrow ones off the left edge in a single step.

Freezing the headings

Layout > Freeze pins the rows above the cursor and the columns to its left, so that they stay on screen while the rest scrolls.

1. Put the cursor on the first cell of the *data* — if the headings are in row 1 and column `A`, that is `B2`.
2. Choose **Layout > Freeze**.

Everything above and to the left of the cursor now stays put. Choosing **Freeze** again unfreezes: it is one command, not two, and being frozen already is what tells it which to do.

NOTE Commander Calc refuses to freeze anything that would leave nothing to scroll. Rows are frozen only if the cursor is in the top half of the visible rows, and columns only if at least one more column of default width would still fit across the screen. If nothing appears to happen, the cursor is too far down or too far right.

There is no split screen, and the freeze is not saved with the workbook.

Using the mouse

If a mouse is connected, Commander Calc shows a pointer and uses it.

Action	Effect
Click in the worksheet	Moves the cell cursor to the cell you clicked.
Click on a worksheet tab	Switches to that worksheet.
Click on the menu bar	Opens the menu under the pointer.
Roll the wheel	Scrolls three rows per notch. Rolling towards you moves down the worksheet.
Click on a row number	Nothing. There is no row selection for it to mean.

While you are typing into a cell the mouse is ignored entirely: clicks do not move the cursor and do not cancel the entry. Finish or cancel the entry first.

Where the cursor is

Three things on the screen tell you, and they never disagree:

- The cell itself is drawn in reverse video.
- Its row number and its column letter are highlighted in yellow.
- Its address appears at the left of the formula bar and again in the middle of the status line.

CHAPTER 5

Entering Data

The two modes

Commander Calc is in one of two states almost all of the time, and the status line says which.

In `READY` (or `MODIF`, which is `READY` with unsaved changes), the keys move the cursor and run commands. In `EDIT`, everything you type goes into a cell.

Typing any ordinary character in `READY` switches to `EDIT` and makes that character the first one of a new entry. There is no key to press first.

CAUTION Typing over a cell replaces it. The old contents are gone the moment you press RETURN — the first keystroke does not append to what was there. To change part of a cell rather than replace it, press F2 instead; see Chapter 6.

Finishing an entry

Five keys store what you have typed, and each one also moves the cursor:

Key	Stores the entry and
RETURN	Moves down one row
↓	Moves down one row
↑	Moves up one row
TAB	Moves right one column
SHIFT + TAB	Moves left one column

ESC abandons the entry. The cell is left exactly as it was.

NOTE ← and → do *not* finish an entry. While you are typing they move the insertion point within what you have typed. This is worth knowing if you are used to a spreadsheet where the left and right arrows commit and move on.

An entry may be up to eighty characters long. The eighty-first keystroke and everything after it is discarded without a sound.

What Commander Calc makes of what you typed

When you finish an entry, Commander Calc decides what kind of value it is by trying five things in this exact order.

Order of interpretation

1. Leading spaces are skipped. If nothing is left, the cell is **cleared**.
2. If what remains is `TRUE` or `FALSE`, in any mixture of upper and lower case, the cell holds a **boolean**.
3. If the *first* character is =, the rest is compiled as a **formula**.
4. If the whole entry reads as a number, the cell holds a **number**.
5. Otherwise the cell holds **text**.

Step 4 is strict: the *entire* entry must be a number. `1.2.3` and `12A` are not numbers, so they become text. This is what keeps a part number from turning into arithmetic.

NOTE Step 3 tests the very first character, before the spaces are skipped. So `=1+1` is a formula and ` =1+1` — with a leading space — is a piece of text that reads `=1+1`. If a cell is showing a formula instead of its answer, that space is the first thing to look for.

Numbers

A number may have a sign, digits, a decimal point, and an exponent introduced by `e` or `E`. Spaces before and after are allowed and ignored.

These are all numbers

```
0
42
-42
+42
123.456
-0.5
.25
8.50
1e3
1.5E2
15e-1
```

These are text

`1.2.3`	*two decimal points*
`12x`	*trailing letter*
`1e`	*no exponent digits*
`-`	*a sign and nothing else*
`007`	*a number, in fact — it becomes 7*

NOTE The last of those catches people out. `007` *is* a valid number and is stored as one; the leading zeros are gone the moment you press RETURN. If you need them, type something that is not a number: a leading apostrophe will not do it, because Commander Calc has no such convention, but a leading space will.

Numbers are stored to about nine significant digits and displayed, in a cell with no format, to nine. A number below 1 is shown without a leading zero — one third is `.333333333`, in the Commodore tradition — and a number of magnitude a thousand million or more is shown in scientific notation. Appendix C explains why, and where the limits are.

Text

Anything the first four steps reject is text. Text may be up to 1,024 characters long, of which the first eighty can be typed — the rest can only arrive from a file.

Text is set to the left of its column, and it is cut off at the column edge if it does not fit. It does not spill over into the empty cell next to it.

TRUE and FALSE

`TRUE` and `FALSE` typed on their own become booleans rather than text, and they are recognised in any case: `true`, `True` and `TRUE` are the same thing. They display as `TRUE` and `FALSE` in capitals whatever you typed.

In arithmetic a boolean counts as 1 or 0, so `=TRUE+TRUE` is 2.

Formulas

An entry beginning with = is a formula. Commander Calc compiles it the moment you press RETURN.

Some formulas

```
=1+1                         2
=A1*2
=SUM(B2:B7)
=IF(C4>0,"over","under")
=Q1!B5+Q2!B5
```

Chapter 17 is the syntax and Chapter 18 is the functions.

NOTE Only = starts a formula. Lotus 1-2-3 users will find that `+A1` and `@SUM(A1:A9)` are not formulas here; they are text.

When a formula will not go in

If the formula does not compile — a syntax error, an unknown function name, a cell reference outside the worksheet, the wrong number of arguments — then Commander Calc **refuses the entry**. The cell keeps whatever it had before.

CAUTION The refusal is silent. No message appears, no error value is stored, and the cursor moves on as though the entry had worked. What tells you is that the cell still holds its old value, or is still empty. Check the formula bar.
The reason for the silence is deliberate: the alternative — storing `=SUM(A1:A9` as a piece of text because it would not compile — would turn a broken formula into a label that looks like it is working.

A formula that compiles but cannot be *evaluated* behaves differently: it goes into the cell and displays an error value such as **`#DIV/0!`**. Those are listed in Chapter 21.

Dates

Commander Calc understands dates, stores them, sorts them and displays them as `YYYY-MM-DD` — but there is no way to type one. Typing `2026-08-08` gives you a piece of text; typing `45000` gives you the number 45,000.

Date cells arrive only from an imported `.XLSX` file or a `.X16S` workbook that already contains them. Chapter 19 explains what happens to them once they are there.

Clearing a cell

Move to it and press BACKSPACE or DELETE. Either one empties the cell completely — value, formula and format. **Edit > Clear** does the same.

Pressing RETURN on an empty entry clears the cell too, which is the long way round to the same place.

CHAPTER 6

Editing

Changing a cell without retyping it

Press F2, or choose **Edit > Edit cell**.

The status line changes to `EDIT`, the formula bar changes colour, and the cell's existing contents appear on it with the insertion point at the end. From there you can move about and change what is there rather than replacing it.

What appears is the cell's *contents*, not its display: a formula cell gives you the formula, and a number formatted as `$8.50` gives you `8.5`.

The edit keys

Key	Effect
←	Insertion point left one character
→	Insertion point right one character
HOME	To the start of the line
END	To the end of the line
BACKSPACE	Delete the character to the left
DELETE	Delete the character under the insertion point
Any printable key	Insert that character
RETURN, TAB, SHIFT + TAB, ↑, ↓	Store the entry and move
ESC	Abandon the entry

Any other key is ignored, and you stay in `EDIT`. Nothing you can press by accident will drop you out of an edit half-finished.

The insertion point is shown by one highlighted character on the formula bar. If the entry is longer than the sixty-eight characters the bar can show, the bar scrolls sideways to keep the insertion point visible.

Undo

Press F5, or choose **Edit > Undo**.

Undo restores the last single cell that changed — the value it had before your last entry or clear.

> **CAUTION** Undo is one cell and one level deep, and *undoing is not itself undoable*. Press F5 twice and the second press does not put things back the way they were before the first; there is nothing to restore.
> Undo does not reach a pasted block, a cleared block, a sort, an inserted or deleted row or column, an import, or a file that has been opened over the top of your work. Sorting has its own undo — see Chapter 11 — and the others cannot be taken back at all.

Copying and pasting

Key	Command
CTRL+A	**Edit > Select** — start or drop a block
CTRL+C	**Edit > Copy** — copies the block, or the current cell
CTRL+V	**Edit > Paste** — pastes with the cursor at the top-left corner

Copying takes what the formula bar would show for each cell — the formula, or the unformatted number, or the text — and pasting enters those into the sheet as though you had typed them, with the cursor cell as the top-left corner of what arrives. Commander Calc says `Copied` when it has taken the block.

To copy one cell, copy with no block marked. To copy a rectangle, mark it first: see Selecting a block below.

- **The format does not travel.** A cell displaying `$8400.00` copied elsewhere arrives as `8400`, in General format. The number is right; the currency is not.
- **References *do* travel.** A pasted formula is rewritten by the distance the block moved — see below.
- **A paste cannot be undone.** F5 restores one cell, and a block is not one cell.

The clipboard survives moving between worksheets, so a block can be copied from one sheet and pasted into another. It does not survive quitting.

Selecting a block

Press CTRL+A, or choose **Edit > Select**. That anchors one corner of the block at the cursor. Now move the cursor: the block grows to whatever rectangle lies between the anchor and wherever you go.

Every movement key extends the selection — the arrow keys, PGUP and PGDN, HOME, END, and a click of the mouse. There is nothing new to learn, because the keys are the ones from Chapter 4.

The anchor may end up below and to the right of the cursor; it makes no difference, and the block is the rectangle between them either way.

To	Press
Start a block	CTRL+A
Grow or shrink it	Any movement key
Drop it	CTRL+A again, or ESC
Copy it	CTRL+C
Clear it	DEL

A command that acts on "the selection" acts on the current cell when there is no block, so CTRL+C and DEL need no block to be useful.

What happens to formulas

A pasted formula does not arrive unchanged. Its references move by the same distance the block did, in both directions at once.

Copy `=A1+B1` five rows down and two columns across, and what lands is `=C6+D6` — which is still the two cells above it, exactly as it was before. Column letters carry properly: one column right of `Z` is `AA`.

To stop a reference moving, write a `$` in front of the part that should stay:

Written	When the formula is copied
`A1`	Both the column and the row move
`$A1`	The column stays at `A`; the row moves
`A$1`	The row stays at 1; the column moves
`$A$1`	Neither moves — always `A1`

That is the ordinary spreadsheet rule, and it is what makes a column of totals worth copying rather than retyping: put `=B2*$C$1` in one cell, copy it down the column, and every row multiplies its own `B` by the one rate in `C1`.

NOTE The same rewriting runs when a formula is carried along by anything else that moves it: inserting a row, deleting a row, sorting, and undoing a sort. Chapter 12 and Chapter 11 describe those.

Two things are deliberately left alone. A reference that names another sheet — `Q1!B5` — is never rewritten, and neither is any formula longer than 128 characters, which is skipped whole rather than half-adjusted.

Clearing

BACKSPACE or **DELETE** in `READY`, or **Edit > Clear**, empties the current cell — or the whole block, if one is marked.

A cleared cell is genuinely empty, not zero. In a range, **SUM** and **AVERAGE** skip it rather than counting it as nothing. Read on its own by a formula, though, an empty cell reads as zero — `=Z90+1` is 1. Chapter 17 sets out the distinction.

CHAPTER 7

Using Menus

The menu bar

The top line of the screen always reads:

```
 File  Edit  Layout  Data  Chart  Sheet
```

Figure 7-1   The menu bar

Six menus, and the division between them is by what a command acts on:

Menu	Holds
File	Everything that touches the storage device.
Edit	The current cell and what is in it.
Layout	The shape of the sheet rather than its contents: which rows and columns exist, how wide they are, and what stays on screen while the rest scrolls.
Data	What is done *to* the numbers: sorting them, and recalculating them.
Chart	The three ways of drawing them.
Sheet	The worksheets themselves.

Opening a menu

Press F10, or click on a menu name.

F10 opens **File**. From there ← and → move between the six menus, wrapping round at either end, and ↑ and ↓ move the highlight through the items, also wrapping. RETURN chooses the highlighted item. ESC closes the menu without doing anything, and so does a second F10.

With the mouse: click a name on the bar to open it, slide along the bar to open each in turn, move down the panel to highlight an item, and click to choose it. Clicking anywhere off the menu closes it.

```
File   Edit   Layout   Data   Chart   Sheet
 New          F1
 Open         F3
 Save         F4
 Save as      F6
 Import       F7
 Export       F8
 Quit         F9
```

Figure 7-2 The **File** menu, with **New** highlighted

NOTE The highlight on the open menu's *name* is four characters wide whatever the name is. **Sheet** therefore appears as `Shee` in reverse video with a stray `t` left in the bar colour beside it, and **Chart** as `Char` with a stray `t`. It is cosmetic, and it is not a sign that anything has gone wrong.

Function keys instead

Every command on **File**, and seven of the eight on **Edit**, has a key printed beside it in the menu, and that key works from the worksheet without opening the menu at all. Most of the time those two menus are there to remind you what the keys are.

The other four menus have no keys at all. Sorting, recalculating, charting, freezing, changing the shape of the grid and every worksheet command can only be reached through the bar. **Replace...** is the one command on **Edit** in the same position, sitting beside seven that do have keys.

NOTE The function keys also work *while a menu is open*. Pressing **F7** with the **Edit** menu on screen closes it and starts an import. Only **PGUP** and **PGDN** are swallowed by an open menu.

The whole menu structure

Chapter 15 describes each of these thirty-four commands in detail.

What is not on the menus

There is no **Help**, no **View** and no **Format** menu. Formatting in particular has no home because there are no formatting commands to put in one — see Chapter 19.

File

`New`	`F1`
`Open`	`F3`
`Save`	`F4`
`Save as`	`F6`
`Import`	`F7`
`Export`	`F8`
`Quit`	`F9`

The storage device.

Edit

`Edit cell`	`F2`
`Select`	`^A`
`Copy`	`^C`
`Paste`	`^V`
`Undo`	`F5`
`Clear`	`DEL`
`Find`	`^F`
`Replace...`	

What is in the cells.

Layout

`Insert row`
`Delete row`
`Insert column`
`Delete column`
`Column width`
`Freeze`

The shape of the grid itself.

Data

`Sort up`
`Sort down`
`Undo sort`
`Recalc now`
`Auto recalc`

What is done to the numbers.

Chart

`Bars`
`Line`
`Pie`

Drawing them.

Sheet

`Next`
`Previous`
`Add...`
`Rename...`
`Delete`

Which worksheet is on screen.

Layout, Data, Chart and Sheet have no keyboard shortcuts at all.

Figure 7-3 Every command on every menu

NOTE The bar has not always had six menus on it. Earlier versions advertised **Cell**, **Sheet** and **Data** with nothing behind them, then dropped back to three that worked; **Data** and **Chart** returned when there was something to put in them, and **Layout** arrived to take the rows, the columns and the widths off **Edit** — which is about editing again as a result. A menu appears here when it has commands, and not before.

CHAPTER 8

Working with Files

A Commander Calc workbook is saved in its own format, `.X16S`, which holds everything: every worksheet, every cell, the formulas as you wrote them, the number formats, the column widths and the worksheet names. Nothing is lost in a save and nothing is lost in a load.

The commands that read and write other programs' files — **Import** and **Export** — are in Chapter 9. This chapter is about your own work.

The file commands

Command	Key
File > New	F1
File > Open	F3
File > Save	F4
File > Save as	F6

Starting a new workbook

F1 throws away what is in memory and gives you one empty worksheet.

If the workbook has been modified, Commander Calc asks `discard unsaved changes?` first, under the title `New workbook`. Y means yes; any other key means no.

Saving

F4 saves. If the workbook already has a name it is written straight back under that name, with no dialogue and no confirmation. If it does not — a workbook you started from **New**, or one that came from an import — you are asked for a name first.

F6, **Save as**, always asks for a name.

The name field takes a name up to twenty-three characters long. Type it, edit it with the same keys you use in a cell, and press RETURN. ESC cancels — and so does pressing RETURN on an empty field.

```
Save workbook as
Name:       BUDGET.X16S
end it .XLSX for Excel, .CSV for plain text
RETURN accept    ESC cancel
```

Figure 8-1   Typing a name for a workbook

NOTE **Save** always writes the native `.X16S` format, whatever the name you give it. The hint about `.XLSX` and `.CSV` on that dialogue belongs to **Export**, which shares the same box. A workbook saved as `ACCOUNTS.XLSX` from **Save** is a `.X16S` file with a misleading name; Excel will not open it and Commander Calc will.

Saving is safe

Commander Calc never writes over your previous workbook until the new one is complete and verified. A save goes:

What a save actually does

1. Write the whole workbook to a temporary file, `CMDRCALC.TMP`.
2. Read it back and check its checksum.
3. Only then delete the old file and rename the temporary one into its place.

If anything fails — a full card, a bad sector, the power going off — the temporary file is removed and your previous workbook is still there, whole. The cost is that a save needs room on the card for two copies of the workbook at once.

Opening

F3 shows a list of the `.X16S` files in the current directory.

```
Open
BUDGET.X16S
INVOICE.X16S
KINDS.X16S
NDQNAT.X16S
RAIN.X16S

up/down select    RETURN open    ESC cancel
```

Figure 8-2   The file list, with the second entry selected

↑ and ↓ move the selection, RETURN opens the file, ESC cancels. With the mouse, the first click on a line selects it and a second click on the same line opens it; a click anywhere off the list cancels.

If the workbook in memory has been modified you are asked `discard unsaved changes?` before the list appears.

CAUTION The list holds at most **thirty-two** files, it is not sorted, and it does not scroll. If the directory contains more than thirty-two `.X16S` files the ones past the thirty-second cannot be reached from this dialogue at all, and there is no indication that they were left out. Keep workbooks in directories of a manageable size.

If there are no matching files at all, Commander Calc says `no matching files on the card` and abandons the command.

What happens when a workbook loads

The header is checked before anything in memory is touched, so a file that is not a `.X16S`, or is a newer version than this program understands, is rejected and your current work is untouched.

Past that point the workbook in memory is cleared and the file is read into it. Formulas are compiled after the file has been closed, which is why a large workbook pauses twice.

NOTE If a file turns out to be damaged *after* the load has started, Commander Calc clears the workbook rather than leaving you with half of one. You get an empty worksheet and a message, not a plausible-looking workbook missing its last four hundred rows.

Naming files

- A name may be up to sixteen characters as far as CMDR-DOS is concerned. The dialogue will let you type twenty-three.
- **Capitals are safest.** Commander Calc converts lower-case letters for the disk system automatically, and converts them back in the file list, so lower case works — but capitals are the range where the machine's character set and the program's agree without translation, and they are what the built-in files use.
- **No extension is ever added.** If you want `.X16S` on the end of the name, type it.
- An empty name cancels the command; it is not an error.

Where files are read and written

Commander Calc uses device 8 for everything. There is no way to change it from the program, and no way to change directory: files are read from and written to whichever directory was current when the program started.

Under the emulator, that is the directory named by `-fsroot`.

The workbook name on screen

Once a workbook has a name, that name appears on the tab line, after the last worksheet tab, for as long as there is room on the line for it. It is the only place the current file name is shown.

What can go wrong

A failed file command puts up a box titled with the command and the word `failed` — `Save failed`, `Open failed` — and one line saying why. The full list of those lines is in Chapter 21; these are the ones you will actually see.

Message	Means
`Disk error`	The card is full, write-protected, or not there.
`File not found`	The name does not exist. From **Open** this usually means the file was deleted between the list being built and your choosing it.
`Damaged or unrecognised file`	Not a `.X16S` file, or one written by a later version of Commander Calc than this.
`Checksum mismatch`	The file is a `.X16S` but its contents do not match its own checksum. Something has corrupted it.
`Out of memory`	The workbook does not fit in this machine's banked RAM.
`Resource limit reached`	The workbook is within memory but past one of the fixed limits — more than 128 styles, more than sixteen worksheets, more than 8,192 distinct pieces of text. See Appendix A.

CHAPTER 9

Importing and Exporting

Commander Calc reads and writes two foreign formats: comma-separated text (`.CSV`) and Excel workbooks (`.XLSX`). Both are interchange formats. Once a file has been imported the workbook lives entirely in Commander Calc's own engine; there is no link back to the file it came from.

Command	Key
File > Import	F7
File > Export	F8

Importing

F7 lists every file in the directory ending `.XLSX` or `.CSV`, in the same thirty-two-entry list **Open** uses. Choose one and Commander Calc decides what to do with it by its name: `.XLSX` goes through the Excel importer, anything else through the CSV reader.

NOTE The box is titled `Open`, not `Import`. It is the same dialogue.

The two importers differ in one large respect: **a CSV is read into the worksheet you are on, at the cursor; an XLSX replaces the entire workbook.**

Importing a CSV

The file is read in at the cursor cell, which becomes the top-left corner of the imported block. Put the cursor on `A1` to import at the origin, or anywhere else to drop the data into a corner of an existing sheet.

Each field is interpreted exactly as though you had typed it into that cell: numbers become numbers, `TRUE` and `FALSE` become booleans, a field beginning = becomes a formula, and `007` becomes the number 7.

The reader accepts	Detail
Commas	And only commas. There is no delimiter detection and no way to ask for semicolons or tabs.
Any line ending	Carriage return, line feed, or both.
Quoted fields	A field starting with `"` runs to the closing quote; `""` inside it is one literal quote.
Rubbish after a closing quote	Discarded rather than treated as an error, because real files contain it.
A missing final newline	The last row still arrives.

An empty quoted field, `""`, leaves the cell empty rather than storing an empty piece of text. Empty fields still take up their column position, so values after a gap stay in the right column.

CAUTION A field is cut off at **ninety-five characters**, silently. Long text in a CSV arrives shortened with no warning of any kind.

When the import finishes, a box reports what happened:

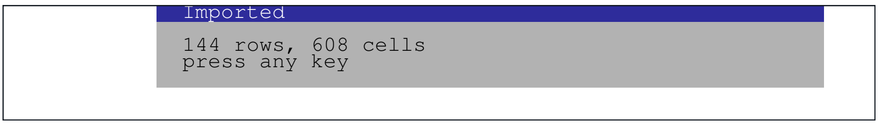


Figure 9-1 The result of a CSV import

If an empty pair of brackets `()` appears after the counts, the import ran off the edge of the worksheet or hit the sixteen-thousand-cell limit and some of the file was left out. The brackets are empty because of a fault in the program — see *Reading the note* below — but they are reliable as a warning sign.

Exporting a CSV

F8 asks for a name. Any name that does not end `.XLSX` produces a CSV.

- Fields are separated by commas and lines end with carriage return and line feed, which is what every other spreadsheet writes.
- A field is quoted only if it needs to be — if it contains a comma, a quote, or a line ending. An embedded quote is doubled.
- The whole used range of the current worksheet is written, empty cells included, so the columns line up.

- **Values are written, not formulas.** A cell holding `=SUM(B2:B7)` exports as `13388.2`.
- Cells are written as they are *displayed*, so a currency-formatted cell exports as `$8400.00`, dollar sign and all.

NOTE Only the current worksheet is exported. A CSV file has no way to hold more than one, so the other fifteen are not written and you are not told.

Importing an XLSX

An `.XLSX` file is a compressed archive of documents. Commander Calc opens the archive, expands each part it needs into memory and reads it — five separate passes, on an eight-megahertz processor, with a trip to the card between each one. It is slow, and it says so:

```
Open
ACCOUNTS.XLSX

Working - please wait.
A large workbook can take a minute or two.
```

Figure 9-2 The busy box shown throughout an import

The box does not animate and does not show progress. It stays exactly as it is until the import finishes.

NOTE For a sense of scale: five hundred days of stock prices in six columns — about three thousand cells — takes **fifty-one seconds** to open as an `.XLSX` and **seventeen seconds** as a `.X16S`. If you are going to use a workbook more than once, import it and then save it in Commander Calc's own format.

What comes across

Feature	Imported
All worksheets	Yes, up to sixteen
Worksheet names	Yes, first 31 characters
Numbers	Yes
Text	Yes, from shared strings or inline
Dates	Yes, as date cells if the style says so
`TRUE` and `FALSE`	Yes
Error values	Yes

Feature	Imported
Number formats	Yes, mapped onto Commander Calc's eight
Column widths	Yes
Formulas	Yes, if Commander Calc can compile them — see below

What does not

Nothing in an `.XLSX` is treated as an error just because Commander Calc does not understand it. Unrecognised content is stepped over, so a workbook with a chart in it still opens — without the chart.

Feature	What happens
Charts, pivot tables	Not read. Gone.
Fonts, fills, borders, colours	Read past. Only the number format is taken; even bold is not imported.
Merged cells	Not handled
Conditional formatting, validation	Not handled
Comments, hyperlinks, images, macros	Not handled
Row heights	Not handled
Frozen panes, print settings, filters	Not handled
Defined names	Not handled
Worksheets past the sixteenth	Skipped, and reported
ZIP64 archives	Refused
Password-protected files	Refused
`.XLS` files	Refused. They are not archives.

Formulas that will not compile

Commander Calc has fourteen functions. An Excel workbook will have formulas it cannot translate — a **VLOOKUP**, a **SUMIF**, anything with a named range.

Those cells are not left empty and are not marked with an error. They keep the value Excel last calculated for them, so the workbook looks right and adds up right. What they will not do is *change* when their inputs change.

The result box says so, with a note in brackets after the counts:

```
Accounts

1284 cells, 96 formulas (some text changed)
press any key
```

Figure 9-3 The result of an XLSX import. The title is the first worksheet's name

CAUTION This is the most dangerous thing in the program, and it is dangerous precisely because it looks like it worked. A cell showing a total that no longer responds to the numbers above it is indistinguishable, on screen, from one that does. After importing an Excel workbook, check the formula bar on the cells that matter.

Reading the note

CAUTION **The counts on a result box are right. The note in brackets after them is not.** The program picks the wrong phrase from its list — it is off by one — so the words that appear describe the *previous* condition, and one case comes out as an empty pair of brackets.
Read the note by what it means here, not by what it says:

What appears	What it actually means
`()`	A limit was reached — cells, formulas, shared strings or styles — and the rest of the file was not imported.
`(some did not fit)`	The file contained characters with no equivalent in the Commander X16's character set. They were replaced with `?`.
`(some text changed)`	Some formulas could not be compiled and are held as fixed values.
No brackets at all	Nothing went wrong.

The phrase `some formulas kept as values` is in the program but can never appear on screen.

The second count is not always the same thing either. If the file had more than sixteen worksheets, the box reports how many were *skipped*; otherwise it reports how many formulas came across.

Reads	Means
`1284 cells, 96 formulas`	All the worksheets fitted; 96 formulas were compiled.
`1284 cells, 2 more sheets skipped`	The file had eighteen worksheets and two were left behind.

When an import fails outright

The workbook in memory is cleared *before* the import begins, so a failure leaves you with an empty workbook rather than a half-replaced one. Save your work before importing.

Message	Means
`Damaged or unrecognised file`	Not a ZIP archive; or the archive has no `xl/workbook.xml`, which means it is not an `.XLSX` at all; or a part did not expand to the size the archive promised.
`Out of memory`	There was not enough banked RAM to stage the file. See below.
`File not found`	The archive is well-formed but does not contain the worksheet its own index points at.

How big a file can be imported

The worksheet's XML has to be held in banked memory in its compressed form *and* its expanded form at the same time, with the growing cell store beside them. One part may be at most 128K expanded.

That is a real limit and it arrives sooner than the cell count does. Five hundred days of six-column data — 118K of XML — imported with two megabytes still free. Two thousand days — 478K of XML — was refused on a machine with 1.98M free. On data of that shape the practical ceiling is around three thousand cells, and a 256K worksheet cannot be imported on a 512K machine at all.

Exporting an XLSX

F8 and a name ending `.XLSX`, in any case.

Every worksheet is written, each as its own part. Text goes inline rather than into a shared string table. Formulas are written twice — as the formula and as the value Commander Calc calculated — so that a program reading the file sees the right number whether or not it recalculates.

Written	Not written
All worksheets and their names	Colours
Numbers, text, dates, booleans, errors	Alignment
Formulas, with their cached values	Frozen panes, row heights
Number formats	Merged cells, charts, defined names

Written	Not written
Bold	A modification date — files show as 1980
Column widths that differ from the default	

NOTE **Nothing is compressed.** Every part of the archive is stored, not deflated, because a DEFLATE compressor would not fit in the space available. A workbook that Excel would write as a 40K file comes out of Commander Calc at about 200K. It is a perfectly valid `.XLSX` — Excel, LibreOffice and Google Sheets all open it — it is simply five times the size.

If an export fails part way through, the file left on the card is not a valid archive at all rather than one that opens with content missing. That is deliberate: a file that will not open is a better outcome than one that opens and is wrong.

CHAPTER 10

Worksheets

A workbook holds up to sixteen worksheets. Each has its own name, its own cells, its own column widths and its own used range, and formulas on any of them can read cells on any other.

All sixteen are saved in one `.X16S` file. There is no way to save one worksheet on its own, or to copy cells from one workbook into another.

The tabs

Line 57 of the screen carries the tabs, ten characters each, the current one in yellow.

Figure 10-1 Three worksheets, with the workbook name after them

A name may be thirty-one characters long, but a tab is ten characters wide and shows only the first nine. Long names are worth avoiding for that reason alone.

After the last tab, if the line has room left, Commander Calc prints the name of the workbook file.

Moving between worksheets

To	Do
Go to the next worksheet	**Sheet > Next**
Go to the previous one	**Sheet > Previous**
Go to a particular one	Click its tab

Next and **Previous** wrap round: **Next** on the last worksheet goes to the first.

> **NOTE** Switching worksheets always puts the cursor on `A1`. It does not remember where you were. This applies to the menu commands and to clicking a tab.

There is no keyboard shortcut for either command.

Adding a worksheet

Sheet > Add... asks for a name and adds a worksheet at the end.

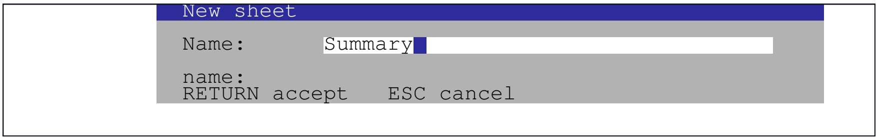


Figure 10-2 Adding a worksheet

The new worksheet is created but *not* switched to. You stay where you were; use **Sheet > Next** or click the new tab to go there.

An empty name cancels. Adding a seventeenth worksheet fails with `Resource limit reached`.

Renaming a worksheet

Sheet > Rename... asks for a new name for the *current* worksheet, with the existing name already in the field so you can edit it.

Renaming a worksheet does not change any formula that refers to it, and a formula naming a worksheet that no longer exists under that name is not re-checked — it keeps its last calculated value. If you rename a worksheet that other worksheets read from, check them.

Deleting a worksheet

Sheet > Delete removes the current worksheet and everything on it.

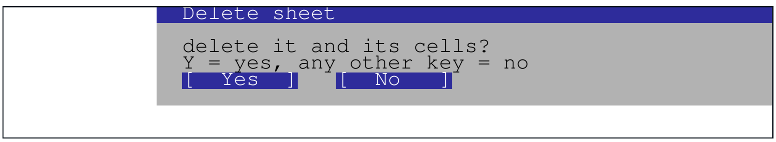


Figure 10-3 Confirming a worksheet deletion

> **CAUTION** There is no undo for this. F5 does not bring a deleted worksheet back; nothing does. The confirmation box is the whole of the safety net.

The last worksheet cannot be deleted — a workbook must have at least one.

Formulas across worksheets

A cell reference may name a worksheet, with an exclamation mark between the name and the reference.

Cross-sheet references

`=Q1!B5`	*cell B5 on the worksheet Q1*
`=SUM(Q1!B2:B40)`	*a range on another worksheet*
`='My Data'!A1`	*quotes are needed for a name with a space*

The name is matched without regard to case and must match the whole stored name. Unquoted names may contain letters, digits, underscores and full stops; anything else — a space, or a leading digit — needs single quotes.

> **NOTE** The prefix applies to exactly one reference and no further. In `='My Data'!A1+A3`, `A1` comes from *My Data* and `A3` comes from the worksheet the formula is on. This trips people up; write `='My Data'!A1+'My Data'!A3` if that is what you meant.

A formula naming a worksheet that does not exist is refused when you enter it, in the same silent way as any other bad reference.

Recalculation is workbook-wide

Every recalculation sweeps every worksheet, not just the one you are looking at. A chain of formulas that runs from *Q1* through *Q2* to *Summary* settles correctly however the cells are ordered and whichever sheet you are on. Chapter 20 explains how.

Sorting

Commander Calc sorts whole rows of the current worksheet into the order given by one column. It is the only command in the program that rearranges more than one cell, and the only one with an undo of its own.

How the range is chosen

There is no dialogue asking what to sort. The cursor decides everything:

- **The column the cursor is in** is the column sorted on.
- **The row the cursor is on** is the first row sorted.
- The sort runs from there down to the last row that has anything in it, across every column that has anything in it.

So to sort a table with a heading row, put the cursor on the *first row of data* — not on the heading — in the column you want to sort by, and the heading stays where it is.

	A	B	C
1	Item	Qty	Price
2	Widget	40	2.50
3	Bolt	12	.75
4	Nut	95	.20
5	Washer	7	.05
6			
7			

Cursor on B2. Heading row is left alone.

	A	B	C
1	Item	Qty	Price
2	Washer	7	.05
3	Bolt	12	.75
4	Widget	40	2.50
5	Nut	95	.20
6			
7			

After **Data > Sort up**.

Figure 11-1 Sorting on column B, ascending

Whole rows move. The `Price` column follows its `Qty`; the table stays consistent.

The commands

Command	Effect
Data > Sort up	Sorts into ascending order
Data > Sort down	Sorts into descending order
Data > Undo sort	Puts the rows back

None of the three has a keyboard shortcut.

After a sort, and after an undo, the cursor is put back on `A1`.

The order values sort in

Values are ranked into four groups, and the groups always come in this order:

Sort order, ascending

1. **Numbers**, smallest first. Text that reads as a number — `456` typed as text — sorts here as a number, not as text.
2. **Text**, alphabetically, ignoring case, on the first thirty-two characters.
3. **Error values**, all equal to each other.
4. **Empty cells**.

Descending reverses the first three groups. It does *not* move the empty rows: blanks stay at the bottom in both directions, which is almost always what you want and is worth knowing when it is not.

A formula sorts by the value it computed, not by the formula text.

Before it sorts

Commander Calc checks three things and may stop or ask.

Condition	What happens
Nothing below the cursor	Nothing happens, and nothing is said.
More than 4,096 rows	`too many rows to sort`
Not enough memory for the index	`not enough memory to sort`
Any formula in the range	`ranges over sorted rows may be wrong,` with Y to go ahead and anything else to stop

A formula carried along by a sort keeps pointing at its own row. A `=B4*C4` sitting on row 4 becomes `=B9*C9` when that row lands on row 9 — so a row that multiplies its own two figures goes on multiplying them, wherever it ends up. That is what a relative reference means, and it is what makes sorting a table with a computed column safe.

CAUTION A *range* is the case the warning is about. `=SUM(B2:B10)` names ten rows that a sort scatters, and afterwards those ten row numbers hold a different ten values. There is no rewriting that can fix it, because the rows the range meant are no longer next to each other. Keep range formulas out of the rows being sorted — in a summary block above the table, or on another worksheet — and sorting will not disturb them.

Undoing a sort

Data > Undo sort reverses the last sort exactly, by replaying the moves backwards.

It works **once**. It refuses — silently — if:

- you have already undone that sort;
- you are on a different worksheet from the one that was sorted;
- the used range has changed size since the sort, because rows have been added or deleted.

Any file command — new, open, save, import, export — forgets the sort as well. Opening a menu does not.

NOTE **Undo sort** and F5 **Undo** are two separate mechanisms that do not interact. F5 will not undo a sort, and **Undo sort** will not undo a cell edit.

CHAPTER 12

Inserting and Deleting Rows

The commands

Command	Effect
Layout > Insert row	Opens a blank row above the cursor's row, pushing everything below it down one
Layout > Delete row	Removes the cursor's row, pulling everything below it up one

Both act on the whole row, across every column that is in use, on the current worksheet only. Neither has a keyboard shortcut.

Inserting

Put the cursor anywhere on the row you want the new row to appear *above*, and choose **Layout > Insert row**.

There is no confirmation. The row appears immediately.

Inserting does nothing if the worksheet is empty, or if the cursor is below the last row in use — there is nothing there to push down. It fails if the last row in use is already row 65,534, because there would be nowhere for the bottom row to go.

Deleting

Put the cursor on the row and choose **Layout > Delete row**. Commander Calc asks first, on the bottom line of the screen:

```
Delete row? (y/n)
```

Figure 12-1 The delete-row question, on the message line

Y deletes the row. Any other key leaves it alone.

CAUTION There is no undo. F5 restores one cell, not one row, and it has no record of a row that has been removed. The question on the message line is the only protection there is.

What happens to formulas

Inserting or deleting a row rewrites the references inside every formula in the worksheet so that they go on meaning the same cells.

Insert a row above row 5

`=SUM(B2:B9)`	`=SUM(B2:B10)` *the range grows*
`=B7*2`	`=B8*2` *the reference follows its cell*
`=B2+B3`	`=B2+B3` *above the cut: unchanged*

Delete row 5

`=SUM(B2:B9)`	`=SUM(B2:B8)` *the range shrinks*
`=B7*2`	`=B6*2`

Ranges come out right whether the row was inside them or was one of their corners. A `$` in a reference makes no difference and is not meant to: a dollar sign fixes a reference against *copying*, and a row that has physically moved has moved for everybody.

The rewrite happens before the recalculation, so the numbers you see afterwards are already the corrected ones.

Three cases it does not cover

CAUTION **A reference that names another worksheet is left alone.** `=Sheet2!A1` still says `A1` after a row is inserted on *Sheet2*. Inserting or deleting rows on a worksheet that others read from means checking those others by hand.
Deleting the row a formula points straight at leaves the formula pointing at that row number, which now holds whatever moved up into it. Excel would put **`#REF!`** there; Commander Calc cannot, because the corrected formula goes back through the compiler and there is no way to write **`#REF!`** that the compiler will accept. The formula keeps working and quietly reads the wrong row.
A formula longer than 128 characters is skipped entirely and keeps its original references. Rewriting it safely needs more working space than the program has, and a formula truncated half way through is worse than one that did not move.

What still does not adjust

CAUTION Two things are left alone wherever a formula moves, and both are deliberate.
A reference naming another worksheet — `Q1!B5` — is never rewritten. Inserting a row on this sheet does not move a reference pointing at that one.
A formula longer than 128 characters is skipped whole rather than half-rewritten. The rewriter works on the source text and has one buffer for it; a formula truncated part way through would be worse than one that did not move.

NOTE Deleting the row a formula points at does not produce `#REF!`. The reference is left pointing at that row number, which now holds whatever moved up into it. Check formulas that referred to a row you have deleted.

What these commands do not do

There is no way to insert or delete a single cell and shift its neighbours, and no way to insert or delete more than one row or column at a time — to open five rows, run **Insert row** five times.

CHAPTER 13

Finding Data

Find

Press CTRL + F, or choose **Edit > Find**.

A prompt appears on the bottom line of the screen, with an underscore marking where the next character will go:

```
Find: Insurance_
```

Figure 13-1 The Find prompt

Type what you are looking for and press RETURN. Commander Calc moves the cursor to the next cell containing it.

Key	Effect
Any printable key	Adds a character to the search text
BACKSPACE	Removes the last character
RETURN	Searches, if there is anything to search for
ESC	Abandons the search

The prompt takes at most **fifteen** characters. It has no insertion point and no arrow keys: you type forwards and delete backwards, and that is all.

What Find matches

- **Anywhere in the cell.** Searching for `sur` finds `Insurance`. It is a substring search, not a whole-cell match.
- **Ignoring case.** `insurance`, `Insurance` and `INSURANCE` all find each other.
- **What the cell displays**, not what it contains. A cell holding `=SUM(B2:B7)` and showing `13388.2` is found by searching for `13388`. Searching for `SUM` will not find it.
- **The current worksheet only.** The other fifteen are not searched.
- **The first 31 characters** of the cell's displayed text. Anything past that is not looked at.

Finding the next one

There is no separate **Find Next** command, and there does not need to be one. Commander Calc looks for the first match that comes *after* the cursor — reading the worksheet row by row, and left to right within a row — and if there is no such match it goes back to the first one anywhere on the worksheet.

Two things follow.

- **Repeating the command walks the matches.** Each search starts from wherever the last one left the cursor, so the cursor steps through every match in turn and then wraps round to the first.
- **The search text is remembered.** The next time you press CTRL + F the prompt already contains what you typed last time, so CTRL + F followed by RETURN is the whole of "find the next one".

CAUTION **A search that finds nothing says nothing.** The prompt disappears and the cursor stays exactly where it was. There is no `not found` message and no sound.
That makes "no match anywhere" and "the only match is the cell I am already on" look identical — in both cases the cursor does not move. If you are not sure which has happened, move the cursor off the cell and search again.

Replace

Choose **Edit > Replace...**. There is no keyboard shortcut.

Commander Calc asks two questions on the bottom line, one after the other:

```
Find: Insurance_
```

```
Replace with: Cover_
```

Figure 13-2 The two Replace prompts

Both take at most fifteen characters and behave exactly like the **Find** prompt — type forwards, BACKSPACE to delete, RETURN to accept, ESC to abandon the whole command.

The replacement may be empty. Press RETURN at the second prompt without typing anything and every match is deleted rather than replaced.

Answering for each cell

Commander Calc does not change anything without asking. It searches the worksheet, and for each cell it finds it moves the cursor there — so you can see what is about to change — and asks:

```
Replace? Y/N/A, Esc to stop
```

Figure 13-3 The question asked at each match

Key	Effect
Y	Replace this one, then go on to the next
N, or any other key	Leave this one alone, then go on to the next
A	Replace this one and **every remaining match**, without asking again
ESC	Stop here. Everything already replaced stays replaced

CAUTION **There is no undo.** F5 does not reach a replacement, and neither does anything else. That is why the command asks cell by cell, and it is worth answering Y a few times before reaching for A.
Save the workbook before a large replacement. If it goes wrong, the way back is to open the saved copy.

What Replace matches

Mostly the same rules as **Find**, with three differences that matter.

- **The whole worksheet, from `A1`.** Unlike **Find**, which starts at the cursor, **Replace** always sweeps the used range from the beginning — left to right along each row, then down. Where the cursor happens to be makes no difference.
- **Formula cells are never touched.** A formula is skipped whether or not its text matches. Searching for `B2` will not rewrite `=SUM(B2:B7)`, and a careless replacement cannot destroy your formulas.
- **Every occurrence within a cell** is replaced, not just the first. Replacing `o` with `0` turns `London Office` into `L0nd0n 0ffice`.
- **Ignoring case**, like **Find** — but the replacement is inserted exactly as you typed it. Replacing `jan` with `Jan` tidies `JAN` and `jan` alike.
- **Numbers as well as text.** What is matched is the cell's contents as the formula bar shows them, so searching a sheet of figures for `222` finds them.

NOTE A replacement can change what kind of cell it is, exactly as retyping would. Replacing `222` with `999` leaves a number; replacing it with `9x9` leaves text, because `9x9` is not a number. The cell is re-read from scratch, not patched.

NOTE A cell whose contents run past 63 characters is skipped rather than truncated, and so is any replacement that would push a cell past that length. Nothing is said when this happens — the cell is simply left alone.

Finding your way around instead

For the common case — getting back to a part of a large worksheet — the navigation keys are usually quicker than Find:

To get to	Press
The top left of the worksheet	SHIFT + HOME
The start of the current row	HOME
A screen at a time	PGUP and PGDN
A particular cell you can see	Click on it

There is no **Go to** command: you cannot type `B400` and be taken there.

CHAPTER 14

Charting

Commander Calc draws three kinds of picture — bars, a line and a pie — all of them of the numbers in one column, and any of them can be saved to the card as a picture file. It is not a charting package. It is the small honest version of the feature that made spreadsheets worth buying, and it takes one command and no configuration at all.

Drawing a chart

Put the cursor in the column you want to chart, on the first row of it, and choose one of the three commands on the **Chart** menu.

Command	Draws
Chart > Bars	A bar chart
Chart > Line	A line chart
Chart > Pie	A pie chart

None of them has a keyboard shortcut, and none has any options. The chart fills the screen. Press S to save it; press anything else to go straight back to the worksheet.

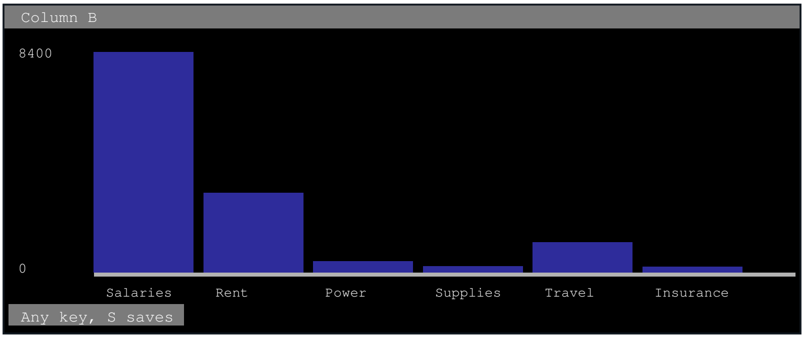


Figure 14-1 **Chart > Bars** on column `B`, with column `A` supplying the labels

NOTE Everything you see on a chart — the bars, the axis figures, the labels, the pie's key — is part of one picture, not text laid over it. That is what makes it possible to save the whole thing, and it is why the letters on a chart are smaller and finer than the ones in the worksheet.

What gets charted

The same rules apply to all three kinds.

- **The cursor's column**, and only that column. There is no way to chart two series together.
- **From the cursor's row downwards**, to the end of the used range.
- **At most sixteen values.** The real limit is how many can be labelled — four characters of screen for each — and on an eighty-column display that works out at sixteen. A longer column is charted from the cursor down and simply stops. To chart the rest, move the cursor further down and run the command again.
- **Numbers only.** Text and `TRUE`/`FALSE` cells are skipped rather than counted as zero — which is what makes it safe to put the cursor on a heading row — and so are empty cells. A formula is charted if its result is a number.

NOTE The chart reads each cell's stored value, not what the worksheet shows. A currency column charts correctly even though what is on screen reads `$8400.00`.

The title line names the column: `Column B`. The bottom of the screen says `Any key, S saves`. If there are no numbers at all from the cursor down, Commander Calc says `No numbers` and waits for a key — the worksheet is not cleared off the screen until the program knows it has something to draw.

The labels

Labels come from **the column immediately to the left** of the one being charted, on the same rows.

So a table laid out with names in column `A` and figures in column `B` charts with its names already in place, which is the layout worth building for. If the cursor is in column `A` there is nothing to the left of it and the chart is drawn without labels.

A label is cut off at the width it has — the width of its bar on a bar chart, twelve characters in the pie's key.

Bars and lines

Both are drawn against the same scale, with the same axis, and differ only in what fills the plot: bars are solid blue blocks, and a line is a red trace with a thicker mark at each value.

- **The scale always includes zero**, whether or not any of the data is near it. A column running from 90 to 100 draws bars that are nearly full height, not a row of stubs differing in the last pixel. A chart whose baseline is not zero exaggerates, and this one is small enough already.
- The top and bottom of the scale are printed up the left-hand edge.
- A zero line is drawn across the plot. On a bar chart **negative values hang below it**.
- A column in which every value is the same draws every bar at full height.

NOTE The line joining two points is a vertical step rather than a true diagonal. At a dozen points across the screen that is very close to what a diagonal would come to anyway, and it costs a great deal less arithmetic on an eight-megahertz processor.

Pie charts

Chart > Pie draws a circle divided into slices in proportion to the values, with a key beside it giving each slice's label and its share as a whole-number percentage.

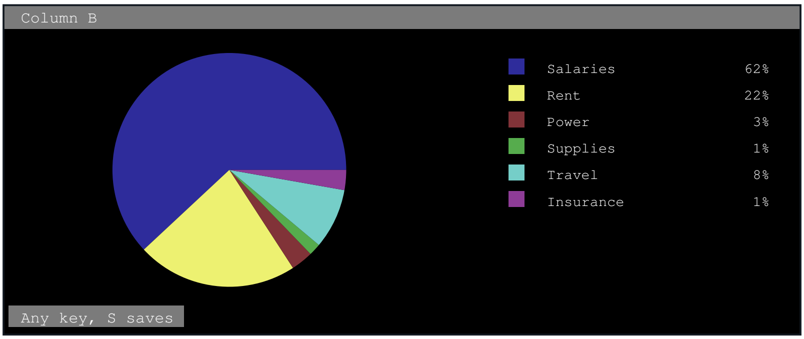


Figure 14-2 **Chart > Pie**, with its key drawn into the picture beside it

- **Negative values are dropped**, not charted as anything — a slice cannot be less than nothing. The percentages are shares of the total of what is left, so a column with negatives in it will not give the shares you expect.
- If every value is negative or zero there is no total to take shares of, and Commander Calc says `No numbers`.
- Percentages are whole numbers, rounded down, so they will not always add up to 100.

- Slice colours cycle through six and then repeat. With more than six values two slices will share a colour; the key is what tells them apart.

Saving a chart

Press S while the chart is on screen.

Commander Calc writes the picture to the card as `CHART.BMP`, returns to the worksheet, and says so on the bottom line:

```
 Saved CHART.BMP
```

Figure 14-3 The message after S

If it cannot be written — a full card, a write-protected one, no card at all — the message is `Could not write CHART.BMP` instead. Either way the message stays until the next time the screen is redrawn.

About the file

Name	`CHART.BMP`, always. You are not asked for one.
Format	Windows `.BMP`, 640 by 480 pixels, sixteen colours.
Size	About 150K.
Read by	Anything that opens a `.BMP` — every paint program, every image viewer, every word processor.

CAUTION **The name is always `CHART.BMP`, so each save overwrites the last one.** If you want to keep two charts, copy or rename the first before drawing the second.

NOTE `.BMP` rather than anything smaller because it is the one picture format this program can write honestly: the others need compression, and Commander Calc carries only the code to *un*compress. The saved file is bigger than it needs to be and completely standard.

It is also twice the height of the picture the machine draws. Commander Calc charts into a bitmap 640 by 240, and shows it on a display running at half vertical scale — so a dot on that bitmap is one pixel wide and two tall, and the pie is swept as an ellipse precisely so that the glass shows a circle. A file has no such display behind it, and every program that opens one assumes square dots. Each row is therefore written twice, which puts the scaling into the file and makes it open as the circle you were looking at.

What a chart is not

- It cannot be printed, and it is not stored in the workbook — neither `.X16S` nor `.XLSX` carries it. `CHART.BMP` is the only record of one.
- There is no stacked, scatter or area chart.
- There is no title you can set, no axis labelling, no colour choice and no scaling option. The pie's key is the only legend there is.
- A chart is a picture of the numbers as they were when you drew it. It does not update, because it does not persist.

PART TWO

Reference

Every command, every key, every function, every message, arranged for looking things up rather than for reading. Each entry stands on its own and says what the program does, not what it ought to do.

CHAPTER 15

Command Reference

Every command on every menu, in alphabetical order. Each entry says what the command does, how to run it, and what it will not do.

Commands are named here as they appear on the menu. **Data > Sort up** is listed under **Sort up**, and the chart commands under their own names — **Bars**, **Line** and **Pie**.

The commands at a glance

Command	Key	Menu
New	F1	File
Open	F3	File
Save	F4	File
Save as	F6	File
Import	F7	File
Export	F8	File
Quit	F9	File
Edit cell	F2	Edit
Select	CTRL + A	Edit
Copy	CTRL + C	Edit
Paste	CTRL + V	Edit
Undo	F5	Edit
Clear	DELETE	Edit
Find	CTRL + F	Edit
Replace...	—	Edit
Insert row	—	Layout
Delete row	—	Layout
Insert column	—	Layout
Delete column	—	Layout
Column width	—	Layout
Freeze	—	Layout

Command	Key	Menu
Sort up	—	Data
Sort down	—	Data
Undo sort	—	Data
Recalc now	—	Data
Auto recalc	—	Data
Bars	—	Chart
Line	—	Chart
Pie	—	Chart
Next	—	Sheet
Previous	—	Sheet
Add...	—	Sheet
Rename...	—	Sheet
Delete	—	Sheet

Alphabetical reference

Add...

Sheet menu. No keyboard shortcut.

Adds a new, empty worksheet to the end of the workbook.

Procedure

1. Choose **Sheet > Add...**
2. Type a name of up to thirty-one characters.
3. Press **RETURN**.

Notes

- The new worksheet is *not* switched to. You stay where you were.
- A tab shows only the first nine characters of a name.
- An empty name cancels the command.
- The seventeenth worksheet fails with `Resource limit reached`.

Auto recalc

Data menu. No keyboard shortcut.

Turns automatic recalculation off, or back on again. It is one command and a switch, not two commands.

Notes

- Commander Calc starts with automatic recalculation on.
- With it off, the right-hand end of the status line reads `CALC` once something has changed and the answers on screen have gone stale. Nothing is shown while they are still current.
- Turning it back on recalculates at once if anything was skipped. Turning it off does nothing immediately.
- See Chapter 20.

Bars

Chart menu. No keyboard shortcut.

Draws the numbers in the cursor's column, from the cursor's row down, as a bar chart filling the screen. **S** saves it as `CHART.BMP`; any other key returns to the worksheet.

Notes

- At most sixteen values. Text, booleans and empty cells are skipped.
- Labels come from the column immediately to the left.
- The scale always includes zero; negative values hang below the zero line.
- `No numbers` if there is nothing to draw.
- See Chapter 14.

Clear

Edit menu. **BACKSPACE** or **DELETE**.

Empties the current cell — its value, its formula and its format.

Notes

- Acts on the marked block if there is one, and on the current cell if not.
- A single cleared cell can be undone with **F5**, once. A cleared block cannot.
- A cleared cell is empty, not zero. **SUM** and **COUNT** skip it; a formula reading it directly gets 0.

Column width

Layout menu. No keyboard shortcut.

Sets the width of the cursor's column, in characters.

Procedure

1. Put the cursor anywhere in the column.
2. Choose **Layout > Column width**.
3. Type a number and press RETURN. ESC cancels.

Notes

- From 3 to 40 characters. The default is 10.
- A worksheet holds at most **nine** widths that differ from the default. Setting a tenth does nothing.
- Widths are saved with the workbook, and are written to and read from `.XLSX`.

Copy

Edit menu. CTRL+C.

Copies the marked block to the clipboard, or the current cell if no block is marked. Commander Calc says `Copied`.

Notes

- What is copied is each cell's *contents* as the formula bar shows them: the formula, or the unformatted number, or the text.
- The number format is not copied.
- The clipboard survives switching worksheets, so a block can be copied from one sheet and pasted into another. It does not survive quitting.

Delete

Sheet menu. No keyboard shortcut.

Deletes the current worksheet and every cell on it.

Procedure

1. Go to the worksheet you want to delete.
2. Choose **Sheet > Delete**.
3. Press [Y] at `delete it and its cells?`

Notes

- **Cannot be undone.**
- The last remaining worksheet cannot be deleted.
- Formulas on other worksheets that referred to this one keep their last calculated values.

Delete column

Layout menu. No keyboard shortcut.

Removes the cursor's column, pulling everything to its right one column left.

Notes

- Asks `Delete column?  (y/n)` first. [Y] goes ahead; anything else stops.
- **There is no undo.**
- Formula references are rewritten so that they go on meaning the same cells.
- Acts on the current worksheet only.

Delete row

Layout menu. No keyboard shortcut.

Removes the cursor's row from the current worksheet and moves everything below it up one.

Procedure

1. Put the cursor anywhere on the row.
2. Choose **Layout > Delete row**.
3. Press [Y] at `Delete row?  (y/n)` on the bottom line.

Notes

- **Cannot be undone.**
- **Formula references are adjusted** — except those naming another worksheet, those pointing straight at the deleted row, and formulas over 128 characters long. See Chapter 12.
- Does nothing if the worksheet is empty or the cursor is below the last used row.

Edit cell

Edit menu. F2.

Puts the current cell's contents on the formula bar for editing, rather than replacing them.

Notes

- The insertion point starts at the end of the line.
- What appears is the contents, not the display: a formula shows as a formula, and a number shows unformatted.
- ESC abandons the edit and leaves the cell untouched.

Export

File menu. F8.

Writes the workbook out in a foreign format.

Procedure

1. Choose **File > Export** or press F8.
2. Type a file name.
3. Press RETURN.

Notes

- A name ending `.XLSX`, in any case, writes an Excel workbook of every worksheet. Anything else writes a comma-separated file of the *current* worksheet only.
- No extension is added for you.
- An exported `.XLSX` is uncompressed and about five times the size Excel would write.
- A CSV export writes displayed values, not formulas.
- Nothing is shown on success.

Find

Edit menu. CTRL+F.

Moves the cursor to the next cell on the current worksheet whose displayed text contains what you type.

Notes

- A substring match, ignoring case, against the cell's *displayed* text — its result, not its formula — and only the first 31 characters of it.
- Searching starts after the cursor and wraps round, so repeating the command walks through the matches. The search text is remembered between searches, which makes CTRL + F RETURN a find-next.
- At most fifteen characters, typed forwards, deleted with BACKSPACE. No insertion point.
- Current worksheet only.
- **A failed search says nothing** and leaves the cursor alone.

Freeze

Layout menu. No keyboard shortcut.

Pins the rows above the cursor and the columns to its left so that they stay on screen while the rest scrolls. Choosing it again unfreezes.

Procedure

1. Put the cursor on the first cell of the data — `B2` if the headings are in row 1 and column `A`.
2. Choose **Layout > Freeze**.

Notes

- Rows are frozen only if the cursor is in the top half of the visible rows; columns only if another column of default width would still fit across the screen. Otherwise that half of the request is refused, silently.
- The freeze is not saved with the workbook.

Import

File menu. F7.

Reads a foreign file into the workbook.

Procedure

1. For a CSV, first put the cursor where you want the top-left corner of the data to land.
2. Choose **File > Import** or press F7.
3. Choose a file from the list and press RETURN.

Notes

- **A CSV is read in at the cursor. An XLSX replaces the whole workbook.** Save first.
- The list shows only `.XLSX` and `.CSV` files, and only the first thirty-two of them.
- The dialogue is titled `Open`.
- Formulas an XLSX contains that Commander Calc cannot compile are kept as their last calculated values and will not update. See Chapter 9.

Insert column

Layout menu. No keyboard shortcut.

Opens a blank column at the cursor, pushing everything from there rightwards one column along.

Notes

- Formula references are rewritten so that they go on meaning the same cells.
- Does nothing if the last column in use is `IU` or later — there would be nowhere for the last column to go.
- Acts on the current worksheet only. Cannot be undone; delete the column again to reverse it.

Insert row

Layout menu. No keyboard shortcut.

Opens a blank row above the cursor's row.

Notes

- No confirmation is asked.
- **Formula references are adjusted**, with the three exceptions in Chapter 12.
- Does nothing if the worksheet is empty or the cursor is below the last used row; fails if the last used row is 65,534.

Line

Chart menu. No keyboard shortcut.

The same values as **Bars**, against the same scale, drawn as a red trace with a thicker mark at each value instead of as blue blocks. S saves it as `CHART.BMP`.

New

File menu. F1.

Discards the workbook in memory and starts an empty one.

Notes

- Asks `discard unsaved changes?` if the workbook has been modified.
- Gives one worksheet, the cursor on `A1`, and no file name.
- A failure here is reported as `Save failed`, which is a slip in the program rather than anything to do with saving.

Next

Sheet menu. No keyboard shortcut.

Goes to the next worksheet, wrapping round from the last to the first.

Notes

- The cursor is put on `A1`. Your position on the old worksheet is not remembered.

Open

File menu. F3.

Loads a `.X16S` workbook.

Procedure

1. Choose **File > Open** or press F3.
2. Answer `discard unsaved changes?` if it is asked.
3. Move the highlight with ↑ and ↓.
4. Press RETURN.

Notes

- The list holds **at most thirty-two** files, unsorted, and does not scroll.
- Only `.X16S` files are listed.
- An invalid file is rejected before the workbook in memory is touched. A file that fails part way through loading leaves an empty workbook.

Paste

Edit menu. CTRL + V.

Enters the clipboard's contents into the current cell.

Notes

- The cursor is the **top-left corner** of what arrives.
- Exactly as though you had typed them — so a formula is compiled again, and a number arrives with no format.
- **A pasted formula is rewritten for its new position**, by the distance the block moved, in rows and columns at once. `=A1+B1` copied five rows down and two across becomes `=C6+D6`. A `$` pins what should not move.
- Not undoable. F5 restores one cell, and a block is not one cell.

Pie

Chart menu. No keyboard shortcut.

Draws the same values as a circle divided into slices, with a key beside it giving each slice's label and its share as a whole-number percentage. S saves it as `CHART.BMP`.

Notes

- **Negative values are dropped**, and the percentages are shares of the total of what is left.
- If nothing is left, `No numbers`.
- Percentages are whole numbers rounded down, so they need not total 100.
- Slice colours cycle through six; with more than six values, two slices share a colour and the key is what tells them apart.

Previous

Sheet menu. No keyboard shortcut.

Goes to the previous worksheet, wrapping round from the first to the last. The cursor is put on `A1`.

Quit

File menu. F9.

Leaves Commander Calc and returns to BASIC.

Notes

- Asks `discard unsaved changes?` if the workbook has been modified.
- Y quits. Any other key cancels.

Recalc now

Data menu. No keyboard shortcut.

Recalculates every formula in the workbook immediately, whether automatic recalculation is on or off.

Notes

- This is what clears `CALC` from the status line.
- With automatic recalculation on it has nothing to do, but it is harmless.

Rename...

Sheet menu. No keyboard shortcut.

Changes the name of the current worksheet. The existing name appears in the field ready to be edited.

Notes

- Formulas on other worksheets that name this one by its old name are not rewritten and are not re-checked. They keep their last values.

Replace...

Edit menu. No keyboard shortcut.

Finds text throughout the worksheet and replaces it, asking at each match.

Procedure

1. Choose **Edit > Replace...**.
2. Type what to find, and press RETURN.
3. Type what to put in its place, and press RETURN. An empty replacement deletes the matched text.
4. At each match the cursor moves there and Commander Calc asks `Replace?  Y/N/A, Esc to stop`.

Notes

- Y replaces this one, N skips it, A replaces every remaining match without asking, ESC stops.
- **The whole used range, from `A1`** — not from the cursor, unlike **Find**.
- **Formula cells are never touched.**
- Every occurrence within a cell is replaced, and matching ignores case.
- Numbers are matched as well as text, and a replacement can change a cell from one to the other.
- **Cannot be undone.** Save first.
- See Chapter 13.

Save

File menu. F4.

Writes the workbook in Commander Calc's own `.X16S` format.

Notes

- If the workbook already has a name it is saved under that name with no dialogue. Otherwise you are asked for one.
- **Always writes `.X16S`**, whatever extension the name has.
- The save is atomic: written to `CMDRCALC.TMP`, verified, and only then renamed over the old file.
- Needs room on the card for two copies of the workbook.
- Nothing is shown on success; the status line changes from `MODIF` back to `READY`.

Save as

File menu. F6.

The same as **Save**, except that it always asks for a name.

Select

Edit menu. CTRL + A.

Anchors one corner of a block at the cursor, or drops the block if one is already marked.

Notes

- Once anchored, **every movement key extends the block** — arrows, PGUP, PGDN, HOME, END, and a mouse click.
- ESC drops the block, and so does CTRL + A again.
- The anchor may end up below and right of the cursor; the block is the rectangle between them either way.
- **Copy** and **Clear** act on the block when there is one and on the cursor cell when there is not.

Sort down

Data menu. No keyboard shortcut.

Sorts rows into descending order on the cursor's column, from the cursor's row down. See **Sort up**; everything there applies, and empty rows still sort to the bottom.

Sort up

Data menu. No keyboard shortcut.

Sorts whole rows into ascending order on the column the cursor is in, starting at the row the cursor is on.

Procedure

1. Put the cursor in the column to sort by, on the first row of data — not on the heading.
2. Choose **Data > Sort up**.
3. If the range contains formulas, press Y at `ranges over sorted rows may be wrong`.

Notes

- Order: numbers, then text (case-insensitive, first thirty-two characters), then errors, then blanks. Blanks stay last in both directions.

- **Relative references move with their row**, so a formula reading its own row goes on reading it. A *range* spanning sorted rows cannot be fixed that way, which is what the warning is about.
- At most 4,096 rows; beyond that, `too many rows to sort`.
- The cursor is put on `A1` afterwards.

Undo

Edit menu. F5.

Restores the last single cell that was changed.

Notes

- One cell, one level. **Undoing is not itself undoable.**
- Does not reach a pasted or cleared block, a sort, an inserted or deleted row or column, a deleted worksheet, an import, or an opened file.

Undo sort

Data menu. No keyboard shortcut.

Puts the rows back the way they were before the last sort.

Notes

- Works once, then not again.
- Refuses, silently, if you are on a different worksheet or the used range has changed size.
- Forgotten by any file command.
- Unrelated to F5 **Undo**.

CHAPTER 16

Key Reference

Every key Commander Calc responds to, arranged by where you are when you press it.

In the worksheet

This is `READY` or `MODIF` mode: the status line says one of those, and you are not typing into a cell.

Moving

Key	Action
↑	Up one row
↓	Down one row
←	Left one column
→	Right one column
RETURN	Down one row
TAB	Right one column
SHIFT + TAB	Left one column
PGUP	Up one screenful (54 rows)
PGDN	Down one screenful
HOME	Column `A` of this row
SHIFT + HOME	Cell `A1`
END	Column `IV` of this row — *not* the last used column

Commands

Key	Action
F1	**File > New**
F2	**Edit > Edit cell**
F3	**File > Open**
F4	**File > Save**

Key	Action
F5	**Edit > Undo**
F6	**File > Save as**
F7	**File > Import**
F8	**File > Export**
F9	**File > Quit**
F10	Open the menu bar
F11	**Edit > Paste** — the same as CTRL + V
F12	**Not used.** It belongs to the machine's debugger.
CTRL + C	**Edit > Copy**
CTRL + V	**Edit > Paste**
CTRL + F	**Edit > Find**
BACKSPACE	Clear the current cell
DELETE	Clear the current cell
ESC	Nothing
Any printable key	Start typing a new entry into the current cell

NOTE The **Data**, **Chart** and **Sheet** menus have no keys at all, and neither do **Insert row** and **Delete row** on **Edit**. Sorting, recalculating, charting, freezing, inserting or deleting a row, and every worksheet command can only be reached through the menu bar.

Keys that share a code

The Commander X16 sends the same character code for certain control combinations as it does for named keys, and Commander Calc cannot tell them apart. Pressing either of a pair does whatever this manual says the named key does.

This	Is the same as	Which does
CTRL + B	PGDN	Down one screenful
CTRL + D	END	To column `IV`
CTRL + I	TAB	Right one column
CTRL + M	RETURN	Down one row
CTRL + P	F9	Quit
CTRL + Q	↓	Down one row
CTRL + S	HOME	To column `A`
CTRL + T	BACKSPACE	Clear the cell
CTRL + U	F10	Open the menu bar

This	Is the same as	Which does
CTRL + X	SHIFT + TAB	Left one column
CTRL + Y	DELETE	Clear the cell

CAUTION CTRL + P quits the program and CTRL + T clears the cell. Neither is what those combinations do in most software, and the first of them will ask to discard your changes if you have any.

While typing into a cell

This is `EDIT` mode. The editor sees every key first, so nothing in this table can be overridden by a command.

Key	Action
Any printable key	Insert that character at the insertion point
←	Insertion point left one character
→	Insertion point right one character
HOME	To the start of the line
END	To the end of the line
BACKSPACE	Delete the character to the left
DELETE	Delete the character at the insertion point
RETURN	Store, and move down
↓	Store, and move down
↑	Store, and move up
TAB	Store, and move right
SHIFT + TAB	Store, and move left
ESC	Abandon the entry
Anything else	Ignored

The entry is limited to eighty characters. Beyond that, keystrokes are discarded silently.

With a menu open

Key	Action
←	Previous menu, wrapping round the five
→	Next menu, wrapping round the five
↑	Previous item, wrapping
↓	Next item, wrapping
RETURN	Choose the highlighted item
ESC	Close the menu
F10	Close the menu
F1–F9, BACKSPACE, SHIFT + HOME	Close the menu and do what that key does in the worksheet
PGUP, PGDN	Ignored

In the file list

Key	Action
↑	Previous file. Does not wrap.
↓	Next file. Does not wrap.
RETURN	Open the highlighted file
ESC	Cancel
First click on a line	Select it
Second click on the same line	Open it
Click off the list	Cancel

In a name box

The full editor is available: arrows, HOME, END, BACKSPACE, DELETE and insertion.

Key	Action
RETURN	Accept the name
ESC	Cancel
RETURN on an empty field	Cancel
Click on the hint line, left half	The same as RETURN
Click on the hint line, right half	The same as ESC

Names are cut off at twenty-three characters for a file and thirty-one for a worksheet.

In a message or confirmation box

Box	Keys
Message (`press any key`)	Any key, or any click, dismisses it
Confirmation	Y means yes. **Any other key means no.** A click on `[ Yes ]` means yes; a click anywhere else means no.

At the Find prompt

Key	Action
Any printable key	Add a character, up to fifteen
BACKSPACE	Remove the last character
RETURN	Search
ESC	Abandon
Arrow keys	Ignored

While a chart is on screen

Key	Action
S	Save the chart to the card as `CHART.BMP`, then go back to the worksheet
Any other key	Go back to the worksheet

S works in either case. See Chapter 14.

At the Delete row question

Y, in either case, deletes the row. Any other key cancels.

CHAPTER 17

Formula Reference

Starting a formula

A cell entry is a formula if, and only if, its *first* character is =.

`=1+1`	`2` *a formula*
`␣=1+1`	`=1+1` *a leading space, so it is text*
`+A1`	`+A1` *text — Lotus notation is not supported*
`@SUM(A1:A9)`	`@SUM(A1:A9)` *text*

Operators

There are twelve, in five levels of precedence.

binds tightest	`−x` `+x` `( )`	unary sign, parentheses
	`^`	power, left to right
	`*` `/`	multiply, divide
	`+` `−`	add, subtract
binds loosest	`=` `<>` `<` `>` `<=` `>=`	comparison, does not chain

evaluated later

Figure 17-1 Operator precedence, tightest binding at the top

Consequences worth knowing

`=2+3*4`	`14` *multiplication first*
`=(2+3)*4`	`20`
`=100/10/2`	`5` *left to right, not 100/(10/2)*
`=2^3^2`	`64` *power is* left-*associative, as in Excel*
`=-2^2`	`4` *the sign binds tighter than the power*
`=1+1=2`	`TRUE` *arithmetic before comparison*

NOTE `=2^3^2` is 64 here and 64 in Excel, but 512 in ordinary mathematical notation and in most programming languages. `=-2^2` is 4 here and 4 in Excel, but −4 in mathematics. Commander Calc agrees with the spreadsheet convention, not the mathematical one, on purpose.

Comparison operators

Written =, <>, <, >, <=, >=. There is no `!=` and no ==. They produce `TRUE` or `FALSE`.

Comparisons do not chain: `=A1<B1<C1` is a syntax error and the entry is refused.

CAUTION **Comparing text does not work properly.** Two cells that both contain `abc` compare as *not* equal, because Commander Calc compares text by identity rather than by content and each piece of text in the workbook is a separate object. `="a"="a"` inside a single formula is `FALSE` for the same reason.

Worse, unequal text compares as “greater” in both directions, so `="a">"b"` and `="b">"a"` are both `TRUE` and < on text is always `FALSE`.

Comparison is only reliable on numbers, dates and booleans. Do not build a worksheet around comparing text.

What is missing

There is no text concatenation operator — no `&`. There is no percent postfix: `=50%` is a syntax error, not 0.5.

Cell references

A reference is one or two column letters followed by one to five row digits: `A1`, `B9`, `AZ100`, `IV65535`. Letters may be upper or lower case.

Columns run `A` to `IV` (1 to 256) and rows 1 to 65,535. Anything outside that — `=A0`, `=ZZ1`, `=A70000` — is an invalid reference and the entry is refused.

Dollar signs: absolute references

A `$` before the column letters, before the row digits, or before both, stops that part of the reference moving when the formula is copied.

Written	When the formula is copied
`B1`	Relative. Both the column and the row move.
`$B1`	The column stays at `B`; the row moves.
`B$1`	The row stays at 1; the column moves.
`$B$1`	Absolute. Always `B1`, wherever it is copied.

This is what makes one formula worth copying down a column. Put `=B2*$C$1` in a cell and copy it down: each row multiplies its own `B` by the single rate in `C1`, because the first reference moves and the second does not.

NOTE The evaluator does not know the difference. The compiler parses `$` and discards it, so `=$B$1` and `=B1` are the same bytecode and give the same answer. The `$` survives in the formula's *source text*, which is what the rewriter reads when a cell moves — and the rewriter is the only thing that needs to know.

One consequence: **a `$` does not protect a reference from an inserted or deleted row**. A dollar sign fixes a reference against copying, and a row that has physically moved has moved for everybody. See Chapter 12.

Ranges

Two references and a colon: `A1:B9`. No spaces are allowed around the colon — `=A1 : B2` is a syntax error, although `=A1 + B2` is fine.

A range is always a rectangle, not a list of cells. `=SUM(B1:C2)` adds four cells.

The corners may be given either way round: `B20:A1` and `A1:B20` are the same rectangle.

CAUTION **Only five functions understand a range**: **SUM**, **AVERAGE**, **MIN**, **MAX** and **COUNT**. Give a range to any other function and you get **`#VALUE!`** — `=ABS(A1:A3)` and `=ROUND(A1:A3,2)` both fail that way.
A range used outside a function collapses to its top-left cell without comment: `=A1:B9*2` quietly means `=A1*2`.

There are no whole-column references (`A:A`), no three-dimensional ranges across worksheets, and no named ranges.

References to another worksheet

Three spellings:

`=Q1!B5`	*plain name*
`=SUM(Q1!B2:B40)`	*a range*
`='My Data'!A1`	*quotes for a name with a space*

An unquoted name may contain letters, digits, underscores and full stops. Anything else — including a space, or a leading digit — needs single quotes. Names are matched without regard to case and must match the whole stored name.

The prefix applies to **one following reference only**. In `='My Data'!A1+A3`, `A3` is on the current worksheet.

A worksheet name that does not exist makes the reference invalid and the entry is refused.

Values in a formula

Numbers

Digits, an optional decimal point, and an optional exponent introduced by `e` or `E` with an optional sign: `42`, `3.14`, `1.5E2`, `15e-1`. At most twenty-three characters.

Text

In double quotes. A doubled `""` inside is one literal quote.

`="hello"`	`hello`
`="say ""yes"""`	`say "yes"`

A string literal in a formula may be at most **forty-eight** characters. Longer than that, or unterminated, is a syntax error.

TRUE and FALSE

Written without quotes, in any case.

Spaces

Only the space character is skipped, and only in certain places: around operators, around parentheses, between a function name and its opening bracket, and around commas. `=  1  +  2 ` is a valid formula.

Tabs are not skipped and neither are spaces inside a range's colon. Anything left over after a complete expression is a syntax error, so `=1 2` is refused.

Argument separator

A comma. Semicolons are not accepted.

How a cell is read by a formula

The cell holds	The formula sees
A number or a date	That number
`TRUE` or `FALSE`	1 or 0
Text	Text. In arithmetic that is **`#VALUE!`**; inside a range it is skipped
An error	The same error, which propagates
A formula	That formula's last calculated value
Nothing at all	**Zero** when read on its own — `=Z90+1` is 1 — but **skipped** when it falls inside a range

That last row is the one to remember. `=AVERAGE(B1:B10)` over three populated cells divides by three, not by ten, which is what makes it agree with Excel.

Limits

Limit	Value
Longest formula that can be typed	80 characters
Longest formula in a file	255 characters
Longest string literal	48 characters
Values live at once during evaluation	10
Compiled size of one formula	320 bytes
Nesting depth	No fixed limit; bounded by the above

NOTE The ten-value evaluation limit is the one you can reach by accident. A function call with eleven or more arguments exceeds it, and the cell shows **`#ERR11!`** — which is the program's way of saying it ran out of room, not that your formula is wrong. Use a range instead of a long argument list: `=SUM(A1:A20)` costs one value where `=SUM(A1,A2,A3,...)` costs twenty.

When a formula is refused

These are found while the formula is being compiled — as you press RETURN. Nothing is stored and the cell keeps what it had. No message is shown.

Cause	Example
Syntax error	`=1+ =(1 =1 2 =A1<B1<C1`
Unknown name or function	`=NOSUCH(1) =SUM`
Invalid cell reference	`=A0 =ZZ1 =Nowhere!A1`
Wrong number of arguments	`=IF(1) =ABS(1,2)`
Too long	Over 255 characters, or over 320 bytes compiled

When a formula produces an error

These go into the cell and are displayed. Chapter 21 lists them all.

Value	Cause
#DIV/0!	Division by zero; **MOD** by zero; **AVERAGE** of nothing
#VALUE!	Text where a number was needed, or a range given to a function that does not take one
#NUM!	Overflow, or an argument out of range
#CYCLE!	Circular reference
#ERR*n*!	A fault in the program rather than in the formula, reported with its own code. **#ERR11!** is the one you can provoke: more than ten live values

CHAPTER 18

Function Reference

Commander Calc has fourteen functions. This chapter documents all fourteen; there are no others, and there is no facility for adding any.

Function names may be typed in any case. `=sum(a1:a9)` and `=SUM(A1:A9)` are the same formula.

The functions at a glance

Function	Arguments	Returns
ABS	1	Absolute value
AND	1 or more	`TRUE` if every argument is non-zero
AVERAGE	1 or more	Mean of the numeric values
COUNT	1 or more	How many numeric values there are
IF	2 or 3	One of two values, by a test
INT	1	Largest whole number not above the value
LEN	1	Number of characters
MAX	1 or more	Largest numeric value
MIN	1 or more	Smallest numeric value
MOD	2	Remainder
NOT	1	`TRUE` if the argument is zero
OR	1 or more	`TRUE` if any argument is non-zero
ROUND	1 or 2	Value rounded to a number of places
SUM	1 or more	Total of the numeric values

Ranges

CAUTION Only **SUM**, **AVERAGE**, **MIN**, **MAX** and **COUNT** accept a range as an argument. Every other function given a range returns **`#VALUE!`** (**LEN** returns 0). There is no exception and no warning.

Inside a range, those five functions ignore empty cells and text, and count `TRUE` and `FALSE` as 1 and 0. An error value anywhere in a range propagates out as the function's result.

Errors in arguments

If any argument evaluates to an error, that error becomes the function's result without the function running — with one exception. **IF** is exempt, so that it can choose the branch that is not broken:

`=SUM(A1,1/0)`	`#DIV/0!` *the error wins*
`=IF(TRUE,1,1/0)`	`1` *the broken branch is not taken*

Alphabetical reference

ABS

`ABS(number)`

Returns the absolute value of *number* — the value with any minus sign removed.

Arguments

- *number* — a number, a boolean, or a reference to a cell holding one. **Not a range.**

Examples

`=ABS(7)`	`7`
`=ABS(-7)`	`7`
`=ABS(-3.25)`	`3.25`
`=ABS(0)`	`0`
`=ABS(A1:A3)`	`#VALUE!` *ranges are not accepted*
`=ABS("x")`	`#VALUE!`

AND

`AND(value1, value2, ...)`

Returns `TRUE` if every argument is non-zero, `FALSE` otherwise.

Arguments

- One or more numbers, booleans or cell references. **Not ranges.**

Notes

- Every argument is evaluated. There is no short-circuiting: a **`#DIV/0!`** in the fourth argument shows up even if the first is already `FALSE`.
- Text in any argument gives **`#VALUE!`**.

Examples

`=AND(TRUE,TRUE)`	`TRUE`	
`=AND(1,1,1)`	`TRUE`	
`=AND(1,0)`	`FALSE`	
`=AND(A1>0,A1<100)`		*the usual use*
`=AND(-1)`	`TRUE`	*non-zero is true, sign and all*

AVERAGE

`AVERAGE(value1, value2, ...)`

Returns the arithmetic mean of the numeric values among the arguments.

Arguments

- One or more numbers, booleans, cell references or ranges, freely mixed.

Notes

- **Empty cells and text are excluded from both the total and the count.** An average over ten cells of which three hold numbers divides by three.
- **AVERAGE** of nothing numeric is **`#DIV/0!`** — unlike **MIN** and **MAX**, which return 0.

Examples

`=AVERAGE(2,4,6)`	`4`	
`=AVERAGE(B2:B7)`		
`=AVERAGE(B2:B7,10)`		
`=AVERAGE(Z1:Z9)`	`#DIV/0!`	*all nine cells empty*

COUNT

`COUNT(value1, value2, ...)`

Returns how many of the values are numbers.

Notes

- Booleans count. Empty cells and text do not.
- Ranges are accepted.

Examples

`=COUNT(B1:B10)`	`3`	*three of the ten hold numbers*
`=COUNT(1,2,3)`	`3`	
`=COUNT("a","b")`	`0`	*text is not counted*
`=COUNT(Z1:Z9)`	`0`	

IF

`IF(condition, then)`   `IF(condition, then, else)`

Evaluates *condition* and returns *then* if it is true, *else* if it is not.

Arguments

- *condition* — anything that is not text. **Any non-zero value is true**, including a negative one. Text gives **`#VALUE!`**.
- *then* and *else* — any values at all, of any type. They do not have to be the same type as one another.

Notes

- If *else* is left out and the condition is false, the result is the boolean `FALSE`.
- The branch that is not chosen is still evaluated, but an error in it is discarded — **IF** is the one function whose arguments' errors do not propagate automatically.
- A range as the condition tests as false.

Examples

`=IF(1>0,"yes","no")`	`yes`
`=IF(0,"yes","no")`	`no`
`=IF(0,"yes")`	`FALSE` *no third argument*
`=IF(B4>B3,2026,2025)`	
`=IF(-1,"yes","no")`	`yes` *non-zero is true*
`=IF("x",1,2)`	`#VALUE!` *text condition*

INT

`INT(number)`

Returns the largest whole number that is not greater than *number*.

Notes

- This is a **floor**, not a truncation. It rounds towards minus infinity, so **INT** of a negative number goes further from zero.
- Not a range.

Examples

`=INT(2.7)`	`2`
`=INT(-2.7)`	`-3` *not* −2
`=INT(5)`	`5`
`=INT(-5)`	`-5`

LEN

`LEN(value)`

Returns the number of characters in *value*.

Notes

- For a number or a boolean, the value is first written out in General format and *that* is measured. `=LEN(1/3)` is 10, because one third displays as `.333333333`.
- For a range, the result is 0.
- `=LEN("")` is 0.

Examples

`=LEN("hello")`	`5`
`=LEN("")`	`0`
`=LEN(1/3)`	`10` *the length of .333333333*
`=LEN(TRUE)`	`4`
`=LEN(A1)`	

MAX

`MAX(value1, value2, ...)`

Returns the largest numeric value among the arguments.

Notes

- Empty cells and text are skipped. Booleans count as 1 and 0.
- Ranges are accepted.
- **MAX of nothing is 0**, not an error — which is what Excel does, and is worth remembering when a column you expected to be full is empty.

Examples

`=MAX(3,9,1)`	`9`
`=MAX(B2:B7)`	
`=MAX(Q1!B2:B4,Q2!B2:B4)`	
`=MAX(Z1:Z9)`	`0` *all empty*

MIN

`MIN(value1, value2, ...)`

Returns the smallest numeric value among the arguments. Everything said about **MAX** applies, including that **MIN** of nothing is 0.

Examples

`=MIN(3,9,1)`	`1`
`=MIN(B2:B7)`	
`=MIN(Z1:Z9)`	`0` *all empty*

MOD

`MOD(number, divisor)`

Returns the remainder after *number* is divided by *divisor*.

Notes

- **The result takes the sign of the divisor**, not of the number. This is Excel's rule, and it is not the rule used by C or by most calculators.
- A divisor of zero gives **`#DIV/0!`**.
- Neither argument may be a range.

Examples

`=MOD(7,3)`	`1`
`=MOD(-1,3)`	`2` *not* −1
`=MOD(1,-3)`	`-2` *the divisor's sign*
`=MOD(10,5)`	`0`
`=MOD(5,0)`	`#DIV/0!`

NOT

`NOT(value)`

Returns `TRUE` if *value* is zero and `FALSE` if it is anything else.

Notes

- Exactly one argument.
- Text or a range gives **`#VALUE!`**.

Examples

`=NOT(TRUE)`	`FALSE`
`=NOT(0)`	`TRUE`
`=NOT(5)`	`FALSE`
`=NOT(A1>10)`	

OR

`OR(value1, value2, ...)`

Returns `TRUE` if any argument is non-zero, `FALSE` if all of them are zero. Everything said about **AND** applies: no short-circuiting, no ranges, text gives **`#VALUE!`**.

Examples

`=OR(FALSE,TRUE)`	`TRUE`
`=OR(0,0,0)`	`FALSE`
`=OR(A1<0,A1>100)`	

ROUND

`ROUND(number)`   `ROUND(number, places)`

Rounds *number* to *places* decimal places.

Arguments

- *number* — a number or a reference. Not a range.
- *places* — a whole number from −9 to 9. If left out, 0. Anything outside that range, or anything that is not a whole number, gives **`#NUM!`**.

Notes

- Rounding is **half away from zero**: 2.5 rounds to 3 and −2.5 rounds to −3. This is not banker's rounding.
- A negative *places* rounds to tens, hundreds and so on.
- **ROUND** changes the *value*. To change only what is displayed you need a number format, which cannot be set from the keyboard — see Chapter 19.

Examples

`=ROUND(3.14159,2)`	`3.14`
`=ROUND(2.5)`	`3`
`=ROUND(-2.5)`	`-3` *away from zero*
`=ROUND(1234,-2)`	`1200`
`=ROUND(8.675,2)`	`8.68`
`=ROUND(1,10)`	`#NUM!` *places out of range*

SUM

`SUM(value1, value2, ...)`

Adds up every numeric value among the arguments.

Arguments

- One or more numbers, booleans, cell references or ranges, in any mixture.

Notes

- Empty cells and text inside a range are skipped. A bare text argument is skipped too.
- Booleans add as 1 and 0.
- An error anywhere inside a range propagates.
- **SUM** of nothing numeric is 0.
- Overflow gives **`#NUM!`**.

Examples

`=SUM(1,2,3)`	`6`
`=SUM(B2:B7)`	
`=SUM(B2:B7,C2:C7,100)`	*mixed freely*
`=SUM(Q1!B2:B40)`	
`=SUM(Z1:Z9)`	`0` *all empty*

Functions that do not exist

For the avoidance of doubt, and because these are the ones people look for first: there is no **SQRT**, **POWER**, **ROUNDUP**, **ROUNDDOWN**, **TRUNC**, **SIGN**, **SUMIF**, **COUNTIF**, **COUNTA**, **VLOOKUP**, **HLOOKUP**, **INDEX**, **MATCH**, **CONCATENATE**, **LEFT**, **RIGHT**, **MID**, **TRIM**, **UPPER**, **LOWER**, **TEXT**, **VALUE**, **TODAY**, **NOW**, **DATE**, **YEAR**, **MONTH**, **DAY**, **WEEKDAY**, **ISERROR**, **IFERROR**, **ISBLANK**, **MEDIAN**, **STDEV** or **PMT**.

Raising to a power is done with the `^` operator, so `=A1^0.5` is a square root. **IF** and comparison operators cover a good deal of what **SUMIF** and **COUNTIF** would be used for, one row at a time.

CHAPTER 19

Number Formats and Dates

Where formats come from

CAUTION **There is no command in Commander Calc that sets a number format.** Not on a menu, not on a key. The eight formats described in this chapter are displayed faithfully when a cell has one, but the only ways for a cell to acquire one are to be imported from an `.XLSX` file that already had it, or to be loaded from a `.X16S` workbook that already had it. Everything you type gets the General format and keeps it.

The same applies to bold, to left and right alignment, and to column widths. The machinery is all there, the file formats carry it, and the display honours it — there is simply no user interface for it yet.

The eight formats

Format	Example value	Displays as
General	8.5	`8.5`
General	1/3	`.333333333`
General	1200000000	`1.2E+09`
Integer	3.7	`4`
Decimal, 2 places	8.5	`8.50`
Currency, 2 places	8.5	`$8.50`
Percent, 1 place	0.125	`12.5%`
Date	45000	`2023-03-15`
Time	0.5	`.5`
Text	8.5	`8.5`

General

The default, and what every cell you type gets.

- Zero displays as `0`.

- Up to nine significant digits, with no trailing zeros. `8.50` typed into a cell displays as `8.5`.
- **No leading zero below 1.** One third is `.333333333` and a half is `.5`. This is the Commodore convention and it is what the ROM's number formatter produces.
- A value of a thousand million or more, or smaller than one hundredth, switches to scientific notation: `1.2E+15`.

Integer

Rounded to a whole number and shown with no decimal point. It rounds rather than truncating, so 3.7 displays as `4`. The stored value is unchanged.

Decimal

A fixed number of decimal places, always shown, up to nine. Unlike General, this format *does* print a leading zero: 0.5 at two places is `0.50`.

Currency

A dollar sign followed by the value at a fixed number of decimal places.

> **NOTE** **The currency symbol is always $.** There is no setting for it, and there are no thousands separators either — eight thousand four hundred displays as `$8400.00`, not `$8,400.00`. A workbook imported from Excel with a pounds or euro format is displayed in dollars.

Percent

The value multiplied by a hundred, shown to a fixed number of places, with a per-cent sign. 0.125 at one place is `12.5%`.

Date

The value read as a day number and shown as `YYYY-MM-DD`. See below.

Time

> **NOTE** **The time format is unfinished.** A cell marked as a time displays its raw number — 0.5 shows as `.5`, not as `12:00:00`. This is deliberate: the choice was to show something visibly wrong rather than something plausibly wrong. If you import a workbook with times in it, the times will look like fractions.

Text

For a numeric cell this displays exactly as General does.

Rounding on display

A format rounds what is shown, not what is stored, and it rounds **half away from zero**: at two places, 8.675 shows as `8.68` and −1.005 shows as `-1.01`.

NOTE The underlying number is binary floating point, so a value that looks like exactly 8.675 may be very slightly under it, and will then round down. If a displayed total is one penny out from a hand-added column, this is why. Appendix C explains the representation. **ROUND** in a formula changes the value itself, which is the fix when the value has to be exact.

Alignment

Unless a cell carries an explicit alignment from a file, Commander Calc decides for itself:

Cell holds	Aligned
A number	Right
A date	Right
Text	Left
`TRUE` or `FALSE`	Left
An error value	Left
A formula	By whatever the formula produced

Column widths

The default width is ten characters. A width may be between three and forty, and is stored per column per worksheet, but — as with formats — there is no command that sets one. Widths arrive from `.X16S` and `.XLSX` files and are saved back out again.

A value too wide for its column is cut off at the column edge, with no `####` marker and no spill into the next cell.

Dates

A date in Commander Calc is a number: the count of days since the beginning of 1900, with 1 being 1 January 1900. The date format is what makes it look like a date.

The display

One format only: **`YYYY-MM-DD`**. There is no day-first or month-first option, no month names, and no locale setting. The reason is that ISO order sorts correctly as text and cannot be misread.

Arithmetic

Because a date is a number, arithmetic on it works: a date cell plus 30 is thirty days later, and one date minus another is the number of days between them.

CAUTION The *result* of that arithmetic is an ordinary cell in General format, so it displays as a day number rather than as a date. `=A1+30` on a date in March 2023 shows `45030`, not `2023-04-14`. There is no way to put a date format on the answer.

The 1900 leap year

NOTE 1900 was not a leap year. Excel believes it was, and every spreadsheet file ever written carries the consequence: day number 60 is a 29 February that never happened, and every date after it is one greater than the true count of days.

Commander Calc **reproduces the error deliberately**, because reproducing it is the only way to agree with the files it has to read. Day 60 displays as `1900-02-29`. Dates after 1 March 1900 are correct in every practical sense; the discrepancy is entirely in the first two months of 1900.

Day number 0 displays as `1900-01-00`, which is not a date at all. It is what the format produces for a value that predates the epoch, and it is a sign that a cell has been formatted as a date by accident.

Typing a date

You cannot. There is no **DATE** function, no **TODAY**, and nothing that parses `2026-08-08` as anything but text. Date cells arrive only from files. This is the largest single gap in the program's handling of dates and it is worth planning around: if a worksheet needs dates in it, put them in the file it is imported from.

CHAPTER 20

Recalculation

When it happens

After every change to every cell — unless you have turned that off.

A workbook with no formulas in it skips the whole business, which is why typing into a plain table of numbers is instant and typing into a worksheet full of totals is not.

How it happens

Commander Calc keeps no record of which formula depends on which cell. Building and maintaining a dependency graph would cost memory the machine does not have, so instead the order of evaluation is *discovered*, by repeated passes.

What a recalculation does

1. Every formula in every worksheet is marked out of date.
2. A pass sweeps them all. A formula whose inputs are all up to date is evaluated and marked done. A formula that reads something still out of date is left alone and its provisional answer thrown away.
3. Step 2 repeats until nothing is out of date — or until a pass changes nothing at all.
4. Anything still out of date after a pass that achieved nothing is marked **`#CYCLE!`**.

Pass 1	`A1 =B1+1`	`B1 =C1*2`	`C1 =5`	C1 is a constant, so B1 can go next time
Pass 2	`A1 =B1+1`	`B1 = 10`	`C1 =5`	B1 settles at 10
Pass 3	`A1 = 11`	`B1 = 10`	`C1 =5`	A1 settles. Nothing left out of date; stop

Shaded cells are up to date at the end of that pass.

Figure 20-1 A three-deep chain settling in three passes

Two useful things fall out of this.

- **The order you typed the formulas in does not matter.** A formula written before the cell it depends on resolves perfectly well; it just takes one more pass.
- **Circular references are detected for free.** A pass that makes no progress means the remaining formulas can only be waiting on each other, which is the definition of a cycle. There is no separate check.

The cost is speed: a chain ten formulas deep takes ten passes over the whole workbook. Deep chains of single-cell dependencies are the slowest thing you can build. A worksheet of a hundred formulas that all read constants recalculates in one pass; a worksheet of a hundred formulas in one long chain takes a hundred.

Turning it off

On a worksheet with a long chain of formulas, recalculating after every keystroke-and-RETURN is the slowest part of typing. Two commands exist for that.

Command	Effect
Data > Auto recalc	Turns automatic recalculation off, or back on. One command for both: it is a switch.
Data > Recalc now	Recalculates immediately, whether the switch is on or off.

Neither has a keyboard shortcut. Commander Calc starts with automatic recalculation on.

Knowing which mode you are in

With the switch off, the right-hand end of the status line says so:

```
 MODIF  B9        516088 bytes free         CALC
```

Figure 20-2 Manual recalculation, with an answer on screen that is out of date

Indicator	Means
`CALC`	The switch is off *and* something has changed. **The answers on screen are stale.**
Nothing	Everything else: either automatic recalculation is on, or it is off and nothing has changed since the last one. Either way the screen is current.

NOTE There is no indicator for "switched off, and up to date". Commander Calc used to show `MANUAL` for that and the word was dropped: it is not a state worth a permanent warning, and it turns into `CALC` on the very next edit anyway.

CAUTION `CALC` means the numbers you are looking at are wrong. They are the answers to the question the worksheet used to ask.
Press **Data > Recalc now** before reading anything off a worksheet showing `CALC`, and before saving one — a workbook saved in that state stores the stale values, and opening it again will not recalculate them until something is changed.

Switching back on

Turning automatic recalculation back on recalculates at once if anything was skipped while it was off. The point of the switch is to defer the work, not to lose it.

Turning it *off* does nothing immediately; it just stops.

Everything is recalculated

The sweep covers every worksheet in the workbook, not just the one on screen. That is what makes a chain running from *Q1* through *Q2* to *Summary* settle correctly no matter which sheet you are editing.

Circular references

A formula that depends on itself, directly or through others, displays **`#CYCLE!`**.

`A1:  =A1+1`	`#CYCLE!`	*direct*
`A1:  =A2 / A2:  =A1`	`#CYCLE!`	*indirect*
`A6:  =SUM(A1:A6)`	`#CYCLE!`	*the range covers its own cell*

Every cell in the cycle is marked, and so is every cell that depends on one. Cells that have nothing to do with the cycle are unaffected and keep their correct values.

NOTE Commander Calc shows **`#CYCLE!`** in the cell. Excel shows 0 and a warning in its status bar; Lotus 1-2-3 showed a `CIRC` indicator. Commander Calc's answer is the most direct of the three: the cell that is wrong is the cell that says so.

There is no iterative calculation. A cycle is always an error, and there is no setting that makes Commander Calc converge on one instead.

Formulas that never recalculate

Two kinds of cell look like formulas and do not behave like them.

- **Formulas imported from an `.XLSX` that Commander Calc could not compile.** They hold the value Excel last calculated. They are not formulas any more, and they will not change. See Chapter 9.
- **Formulas the program cannot reach.** If one of the program's files is missing from the card, recalculation cannot run at all and every formula keeps its previous value, silently. See Chapter 2.

In both cases the formula bar is the way to tell: a cell that is still a formula shows its formula there.

CHAPTER 21

Messages and Error Values

Everything Commander Calc can say, in two lists: the error values that appear inside a cell, and the messages that appear in a box or on the status line.

Error values in cells

These appear in the cell itself, aligned to the left like text. There are six with names, and one catch-all that carries a number.

`#CYCLE!`

Cause

A circular reference: the formula depends on itself, directly or through a chain of other formulas. Every cell in the cycle and everything downstream of it is marked.

Remedy

Find the loop and break it. Common cases are a total that includes its own cell — `=SUM(A1:A6)` typed into `A6` — and two cells that read each other. Chapter 20 has more.

`#DIV/0!`

Cause

Division by zero. Also **MOD** with a divisor of zero, zero raised to a negative power, and **AVERAGE** over a range with no numbers in it.

Remedy

Guard the division: `=IF(B1=0,0,A1/B1)`. Note that an empty cell reads as zero when a formula divides by it, so a divisor that has simply not been filled in yet produces this too.

#ERR*n*!

Cause

Something went wrong inside the program rather than inside your formula, and *n* is the internal code for what. This is the catch-all: any error that has no **#...!** spelling of its own is shown this way, with its number, so that the cause can be looked up rather than guessed at.

Shows	Means
#ERR10!	Out of memory — the banked allocator is exhausted.
#ERR11!	A resource limit. During evaluation this means the formula needed more than ten values live at once: a function call with eleven or more arguments, or a deeply nested expression.
#ERR40!	A syntax error reached the evaluator, which should not happen.

Remedy

#ERR11! is the one you can fix yourself: use a range instead of a list of arguments. `=SUM(A1:A20)` needs one value where `=SUM(A1,A2,...,A20)` needs twenty. Or split the formula across two cells. For any other number, save the workbook under a new name and restart.

#NAME?

Cause

A name the program does not know. **This cannot be produced by typing**: a formula with an unknown function in it is refused at entry rather than stored. It appears only in a cell imported from an `.XLSX` file that already contained **#NAME?**.

Remedy

Fix it in the original file, or retype the cell here.

#NUM!

Cause

A number out of range. Arithmetic overflowed — the largest number Commander Calc can hold is about 1.7×10^{38} — or **ROUND** was given a number of places outside -9 to 9, or a negative number was raised to a fractional power.

Remedy

Scale the numbers down. A worksheet of very large values can often be held in thousands or millions instead. Appendix C gives the exact limits.

`#REF!`

Cause

An invalid reference. Like **`#NAME?`**, this **cannot be produced by typing** — a bad reference is refused at entry. It appears only in cells imported from an `.XLSX` that already contained **`#REF!`**.

> **NOTE** Deleting a row in Commander Calc does *not* produce **`#REF!`** in formulas that pointed into it. They quietly point at whatever moved up into its place. That is worse than an error value, and it is why Chapter 12 says what it says.

`#VALUE!`

Cause

The wrong kind of value. Usually one of:

- Arithmetic on a cell containing text — `=A1*2` where `A1` holds `n/a`.
- A range given to a function that does not take one. Only **SUM**, **AVERAGE**, **MIN**, **MAX** and **COUNT** accept ranges; `=ABS(A1:A3)` is **`#VALUE!`**.
- Text as the condition of **IF**, or as an argument to **AND**, **OR**, **NOT**, **ABS**, **INT**, **ROUND** or **MOD**.

Remedy

Check the cells the formula reads. A number that was typed with a stray character in it is stored as text and looks identical in a narrow column.

Messages in boxes

These appear as the body of a box titled with the command that failed — `Save failed`, `Open failed`, `Import failed`, `Export failed`, or `Sheet`.

Message Cause and remedy

Message	Cause and remedy
`Checksum mismatch`	The file is a `.X16S` but its contents do not match the checksum stored in it. Something has damaged it. There is no repair; use a backup.
`Circular reference`	A formula depends on itself. See **`#CYCLE!`** above.
`Damaged or unrecognised file`	Not a file of the type expected: not a `.X16S`, not a ZIP archive, or a `.X16S` written by a later version of Commander Calc than this one.
`Disk error`	The card is full, write-protected, absent, or faulty. On a save, check there is room for two copies of the workbook.
`Division by zero`	As **`#DIV/0!`** above, reported from a context that uses a box.
`File not found`	The name does not exist on the device.
`Formula syntax error`	The formula could not be parsed.
`Import stopped at a limit`	The import ran into one of the fixed limits and did not finish. What did arrive is in the workbook.
`Invalid cell reference`	A reference outside `A1–IV65535`, or naming a worksheet that does not exist.
`Number out of range`	As **`#NUM!`** above.
`Out of memory`	The machine's banked RAM is exhausted. A larger workbook needs a machine with more memory; the status line at startup says how much this one has.
`Resource limit reached`	A fixed limit rather than memory: more than 16,384 cells on a worksheet, 128 styles, sixteen worksheets, 8,192 distinct pieces of text. Appendix A lists them all.
`Unexpected end of file`	The file stops earlier than its own contents say it should. It has been truncated.
`Unknown name or function`	A function Commander Calc does not have. Chapter 18 lists the fourteen it does.
`Unsupported feature`	The file uses something this program does not implement.
`Wrong value type`	Usually the wrong number of arguments to a function — `=IF(1)` or `=ABS(1,2)`.
`Unknown error`	Should not happen. If it does, the workbook in memory is of uncertain soundness; save it under a new name and check it.

Other messages

Message	Where and why
`A large workbook can take a minute or two.`	In the busy box during an import or export. Not an error.
`delete it and its cells?`	Confirming **Sheet > Delete**.
`Delete row?  (y/n)`	On the message line, confirming **Layout > Delete row**.
`discard unsaved changes?`	Confirming **New**, **Open** or **Quit** with a modified workbook.
`end it .XLSX for Excel, .CSV for plain text`	A hint on the name box. It applies to **Export**; **Save** always writes `.X16S` whatever you type.
`ranges over sorted rows may be wrong`	Before a sort of rows containing formulas. Y to go ahead. Relative references move with their rows; a range spanning the sorted rows cannot. See Chapter 11.
`Copied`	The block, or the cell, is on the clipboard. Not an error.
`Delete column?  (y/n)`	Before **Layout > Delete column**. There is no undo for it.
`Any key, S saves`	On a chart. Not an error: S writes it out, anything else goes back to the worksheet.
`no matching files on the card`	The file list found nothing of the right extension in this directory.
`No numbers`	A chart command found nothing to draw from the cursor down. On a pie it also means every value was negative or zero.
`Saved CHART.BMP`	S was pressed on a chart and the picture was written to the card. Not an error.
`Could not write CHART.BMP`	S was pressed and the card is full, write-protected or absent.
`not enough memory to sort`	The sort needs an index in banked RAM and there was none free.
`press any key`	Under every message box.
`too many rows to sort`	More than 4,096 rows in the sort range.
`Working - please wait.`	In the busy box. It does not animate and does not show progress.
`Y = yes, any other key = no`	Under every confirmation box, and meant literally.

On the status line

Message	Means
`READY`	Waiting for input; no unsaved changes.
`MODIF`	Waiting for input; there are unsaved changes.
`EDIT`	You are typing into a cell.
n `bytes free`	Banked RAM available, measured once at startup.
`CALC`	At the right-hand end: automatic recalculation is off *and* something has changed. **The answers on screen are stale.** Use **Data > Recalc now**.
`STACK!`	See below.

STACK!

CAUTION This replaces the free-memory reading on the status line and means that Commander Calc has used more of its internal working stack than it has. The program detected it rather than crashing, but from that point on nothing is guaranteed.
Save the workbook **under a new name** — not over your existing file — and restart the program. Then check the saved workbook before trusting it.

Silence

Several things fail without saying anything at all. They are collected here because a message you do not get is harder to look up than one you do.

You do	And nothing happens because
Type a formula with a mistake in it	The formula would not compile, so the entry was refused and the cell kept its old contents. Chapter 5.
Press CTRL + F and search	Nothing matched. There is no `not found` message. Chapter 13.
Choose **Data > Sort up**	There is nothing below the cursor to sort. Chapter 11.
Choose **Data > Undo sort**	You are on a different worksheet, or the used range has changed, or the sort has already been undone. Chapter 11.
Choose **Layout > Insert row**	The worksheet is empty, or the cursor is below the last used row. Chapter 12.
Choose **Layout > Freeze**	The cursor is too far down the screen, or too far right, for anything to be left to scroll. Chapter 4.

You do	And nothing happens because
Open a menu, or run any file command	One of the program's files is missing from the card. Chapter 2.
Type an eighty-first character into a cell	The entry buffer is full. Chapter 5.

APPENDICES

Technical Reference

The numbers behind the program: what it can hold, what a workbook file contains byte for byte, how a number is stored, and where the machine's memory goes. Nothing here is needed to use Commander Calc. All of it is needed to trust it.

APPENDIX A

Limits and Capacities

Every fixed limit in Commander Calc, in one place. These are compiled into the program; none of them can be raised without rebuilding it.

The worksheet

Limit	Value	Reaching it gives
Worksheets per workbook	16	`Resource limit reached`
Columns per worksheet	256 (`A–IV`)	`Invalid cell reference`
Rows per worksheet	65,535	`Invalid cell reference`
Cells per worksheet	**16,384**	`Resource limit reached`
Last addressable cell	`IV65535`	
Worksheet name	31 characters	truncated
Worksheet name shown on a tab	9 characters	truncated

NOTE The cell limit is per *worksheet*, not per workbook. Sixteen worksheets of 16,384 cells each is 262,144 cells in one file — if the machine's memory stretches that far, which on a 512K machine it does not.

Cell contents

Limit	Value	Reaching it gives
Text in a cell	1,024 characters	truncated
Text that can be typed	80 characters	further keys ignored
Distinct pieces of text in a workbook	8,192	`Resource limit reached`
Number formats (styles) in a workbook	128	`Resource limit reached`
Column width	3 to 40, default 10	clamped silently

Identical pieces of text are *not* shared: two cells both holding `Total` use two of the 8,192.

Formulas

Limit	Value	Reaching it gives
Formula that can be typed	80 characters	further keys ignored
Formula in a file	255 characters	`Resource limit reached`
Compiled size of a formula	320 bytes	`Resource limit reached`
String literal in a formula	48 characters	`Formula syntax error`
Number literal in a formula	23 characters	`Formula syntax error`
Values live during evaluation	10	**`#ERR11!`**
Nesting depth	no fixed limit	

Numbers

Property	Value
Representation	5-byte binary floating point
Significant digits	about 9
Largest magnitude	about 1.7×10^{38}
Smallest non-zero magnitude	about 2.9×10^{-39}
Digits accepted when parsing	11 significant
Rounding	half away from zero
Infinity, not-a-number, negative zero	none

Appendix C describes the representation.

Files

Limit	Value	Notes
File name	16 characters	CMDR-DOS; the box accepts 23
Files listed in a dialogue	**32**	unsorted, does not scroll
Device	8	not changeable
CSV field on import	95 characters	truncated silently
Rows in one sort	4,096	`too many rows to sort`
Find text	15 characters	further keys ignored
Values in one chart	16	the rest are not drawn
Chart slice colours	6	cycled, so slices repeat

XLSX import

Limit	Value	Reaching it gives
One archive part, expanded	128K	`Resource limit reached`
Shared strings mapped	1,024	`(some did not fit)`
Formulas compiled per import	512	the rest keep cached values
Custom number formats	32	the rest read as General
One cell's raw value text	512 bytes	truncated
Archive entry name	48 characters	the part is not found
Worksheets	16	`n more sheets skipped`

NOTE The 128K part limit is the one that bites first, and it is about the *file*, not the worksheet. A worksheet's XML has to be held compressed and expanded at the same time, with the growing cell store beside it. Measured: 500 days of six-column data (118K of XML) imported with 2M free; 2,000 days (478K) was refused with 1.98M free. On data of that shape the ceiling is around three thousand cells.

Memory

Item	Size
Banked RAM on a standard machine	512K (64 banks of 8K)
Banked RAM, maximum	2M (256 banks)
Available to the workbook	All of it except bank 0
Largest single allocation	one bank, 8,192 bytes
A cell record	8 bytes
Row index per worksheet	1K, allocated once
Each 256-row band touched	2K
Resident program	\$0801–\$7FFF
C stack	512 bytes

Storage is sparse but banded. A worksheet using rows 1–100 costs 3K; a worksheet with one cell in each of 256 scattered bands costs 513K. The 16,384-cell limit is what bounds that case.

Undo

Undo	Depth
Edit > Undo (F5)	one cell, one level, not itself undoable
Data > Undo sort	one sort, once, same worksheet only
Everything else	none

APPENDIX B

The .X16S File Format

This appendix describes Commander Calc's native workbook format precisely enough to write a reader for it. Nothing here is needed to use the program.

All integers are little-endian, fixed width, with no alignment and no padding.

The envelope

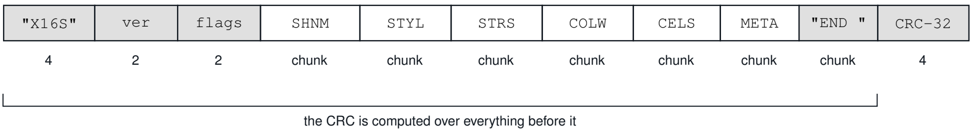


Figure B-1 The layout of a `.X16S` file

Offset	Size	Field
0	4	Magic, the four bytes `X16S`
4	2	Version. Currently 3. A higher number is refused with `Damaged or unrecognised file`.
6	2	Flags. Always written as 0; read and discarded.
8	...	The chunks, in the order shown above
...	8	The terminating chunk: tag `END ` and length 0
...	4	CRC-32 of every byte before this field

Chunks

Every chunk has the same three-part shape:

Offset	Size	Field
0	4	Four-character tag
4	4	Length of the payload, not counting these eight bytes
8	*length*	Payload

A reader must skip a chunk whose tag it does not know rather than rejecting the file. That is the point of the framing, and it is what lets a later version of Commander Calc add to the format without breaking this one.

Chunks are written in a fixed order, and a reader may rely on `STRS` preceding `CELS`, because a text cell names its string by an identifier assigned in `STRS`.

SHNM — worksheet names

The names, NUL-terminated, one after another, in worksheet order. Each is at most 31 characters plus its terminator.

STYL — number formats

Offset	Size	Field
0	2	Count of style records
2	5 each	The style records

Each record is five bytes:

Offset	Size	Field
0	1	Number format: 0 General, 1 Integer, 2 Decimal, 3 Currency, 4 Percent, 5 Date, 6 Time, 7 Text
1	1	Foreground colour
2	1	Background colour
3	1	Flags: bit 0 bold; bits 1–2 alignment (0 automatic, 1 left, 2 right, 3 centre)
4	1	Decimal places

The chunk length must be exactly $2+5n$ or the file is rejected. Style 0 is written but not read back: the reader has already created a General style at index 0 and skips the file's first record.

STRS — the text pool

Offset	Size	Field
0	2	Count of string records
2	…	The records

Each record is a 2-byte identifier, a 2-byte length, and that many bytes of text. Identifiers start at 1 and are the identifiers the cells refer to. A string longer than 1,024 bytes makes the file invalid.

NOTE Text is stored as plain bytes in the machine's own character set, not as PETSCII and not as UTF-8. What is in the file is what appears on screen.

COLW — column widths

One chunk per worksheet: a 1-byte worksheet number followed by 256 width bytes. All 256 are always written whether or not they differ from the default of ten. A short chunk is read as far as it goes; a long one has its tail ignored.

CELS — the cells

One chunk per worksheet.

Offset	Size	Field
0	1	Worksheet number
1	4	Total cell count
5	…	Row blocks, until the chunk ends

Each row block is a 2-byte row number, a 2-byte count of cells in that row, and that many cell records. Rows ascend; within a row, columns ascend. A count of 0, or of more than 256, makes the file invalid.

META — summary

Four 16-bit fields: worksheet count, active worksheet, highest row used, highest column used.

NOTE `META` is written but **not read**. The worksheet count comes from `SHNM` and the used range is recomputed as the cells are loaded. It is recorded here because it is in the file, not because it matters.

The cell record

Eight bytes.

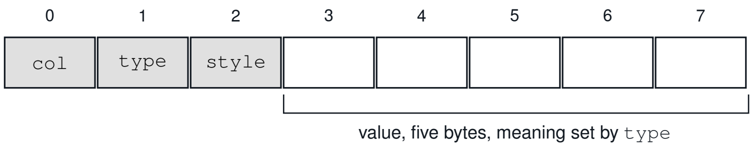


Figure B-2 The eight-byte cell record

Offset	Size	Field
0	1	Column, 0–255
1	1	Type: 0 empty, 1 number, 2 text, 3 boolean, 4 date, 5 formula, 6 error
2	1	Style, an index into `STYL`
3	5	Value

Type	The five value bytes hold
Number	The number, in the 5-byte format of Appendix C
Date	The same, holding a day count
Boolean	The same, holding 1 or 0
Text	A 2-byte string identifier, then three zero bytes
Formula	A 2-byte string identifier naming the formula's *source text*, then three zero bytes
Error	The error code in the first byte

A blank cell has no record at all; storage is sparse.

Version history

Version	Meaning
1	One unnamed worksheet. `COLW` and `CELS` carry no worksheet byte. Still opens; its formulas are dropped.
2	Every worksheet. `COLW` and `CELS` gain the leading worksheet byte. Still opens; its formulas are dropped.
3	A formula cell stores the identifier of its source text, so formulas survive a save and load. Current.

Versions 1 and 2 stored a memory address in a formula cell, which means nothing once the file has been read into a different machine. Those cells are read and discarded.

Saving is atomic

A save writes the whole workbook to `CMDRCALC.TMP`, reads it back and verifies the CRC, and only then deletes the previous file and renames the temporary one over it. Any failure removes the temporary file and leaves the previous workbook untouched. The cost is that the card must have room for two copies at once.

Loading

The header is validated before anything in memory is touched, so a bad header leaves the current workbook alone. Past that point the workbook is cleared first; a failure after that point clears it again, so a damaged file leaves an empty workbook rather than a half-built one.

Formulas are collected as the file is read and compiled after it has been closed, which is why a large workbook pauses twice.

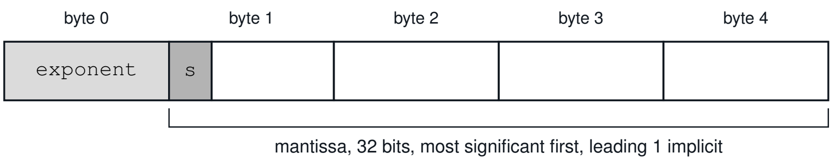
APPENDIX C

How Numbers Are Stored

The format

A number in Commander Calc occupies five bytes and is binary floating point — not fixed point, not binary-coded decimal, and not a scaled integer.

The format is the one the Commander X16's ROM arithmetic library uses. That is the whole reason for the choice: an addition costs a jump into ROM bank 4 rather than a software floating-point library that would not fit in the program.



byte 0 exponent, biased by 128. **Zero means the value is exactly zero.**

bit s the sign, 1 for negative. It sits where the mantissa's always-1 leading bit would be.

value $\pm 0.1mmmm\ldots_2 \times 2^{(\mathrm{exponent}-128)}$

Figure C-1 The five bytes of a number

Three worked examples:

Value	**Bytes**
1.0	`81 00 00 00 00`
5.0	`83 20 00 00 00`
9.81	`84 1C F5 C2 8F`

Range and precision

Property	Value
Significant decimal digits	about 9
Largest magnitude	just under 2^{127}, about 1.70×10^{38}
Smallest non-zero magnitude	about 2.94×10^{-39}
Infinity	does not exist
Not-a-number	does not exist
Negative zero	does not exist
Underflow	silently becomes exactly zero

Nine digits is the honest figure. Thirty-two mantissa bits are worth about 9.6 decimal digits, and the display and the program's own tests both treat nine as the number that can be relied on.

> **CAUTION** **Large integers are not exact.** A figure like a share volume of 7,443,220,000 has more significant digits than the format can hold, and lands on a neighbouring value — it will come back looking like a transcription error. The remedy is to scale: store volumes in thousands, money in whole units, and let a number format put the zeros back.

Binary, not decimal

> **NOTE** This is binary floating point, so 0.1 plus 0.2 behaves the way it behaves everywhere else: the result is very slightly not 0.3, and `=A1+A2=0.3` can be `FALSE` when the cells display `.1` and `.2`.
>
> Number formats round on display, so a column of money *looks* right; and **ROUND** in a formula makes the value itself right. Comparing two computed values for exact equality is the thing to avoid. Compare a difference against a tolerance instead: `=ABS(A1−A2)<0.001`.

Overflow, and why the checks are strict

> **NOTE** The ROM's arithmetic library handles overflow and division by zero by jumping into BASIC's error handler — and there is no return from that. BASIC would print its own error, drop to `READY`, and your unsaved workbook would be gone.

So Commander Calc range-checks every operation *before* calling the ROM, from the exponent bytes alone. The checks are deliberately conservative: they may refuse one value right at the very top of the range that would in fact have fitted. A spurious **#NUM!** on an absurd number costs nothing. A crash costs the workbook.

Operation	Refused when
Add, subtract	Either operand has the maximum exponent — magnitude 8.5×10^{37} or more — even if the result would fit
Multiply	The exponents sum past the top of the range
Divide	The divisor is zero (**#DIV/0!**), or the quotient would overflow
Power	Zero to a negative power (**#DIV/0!**); a negative base with a fractional exponent (**#NUM!**)
MOD	The divisor is zero

x^0 is 1, including 0^0.

Rounding

Operation	Rounds
Arithmetic	In the ROM, using a 40-bit accumulator and a guard byte
ROUND, and every number format	Half away from zero: 2.5 to 3, −2.5 to −3. Not banker's rounding
INT	Down, towards minus infinity: −2.7 to −3
Conversion to a whole number	Towards zero

Reading a typed number

At most **eleven** significant digits are taken into the mantissa. Further integer digits still scale the magnitude; further fraction digits are discarded.

A decimal exponent below −300 gives **#NUM!**. The decimal scaling is applied one factor of ten at a time through the same guarded multiply and divide, so `1E999` reports **#NUM!** rather than crashing the machine.

Writing a number out

In General format the number goes through the ROM's own formatter, with the leading space that CBM machines reserve for a sign stripped off.

Value	Displays as
0	`0`
8.50	`8.5` — trailing zeros are not kept
1/3	`.333333333` — no leading zero
-0.5	`-.5`
1.2×10^{15}	`1.2E+15`

Scientific notation takes over at a thousand million and above, and below one hundredth. The buffer for a formatted number is 24 bytes.

APPENDIX D

Memory

How much there is

The Commander X16 ships with 512K of banked memory and can be expanded to 2M. Commander Calc uses all of it for the workbook, and the figure on the status line at startup says how much that is on your machine.

Machine	Free at startup
Standard, 512K	about 516,000 bytes
Fully expanded, 2M	about 2,097,000 bytes

That figure is a statement of capacity, not of use. It is measured once and does not fall as the workbook grows.

What a workbook costs

Item	Costs
One cell, of any kind	8 bytes
One worksheet, whether used or not	1K
Each block of 256 rows you put anything in	2K
A piece of text	its length, plus a few bytes
A formula	its text, and its compiled form

The third row is the one that catches people out. Storage is allocated in bands of 256 rows, and a band costs the same whether it holds one cell or two hundred and fifty-six.

> **NOTE** A worksheet using rows 1 to 100 costs about 3K. A worksheet with a single cell scattered into each of 256 different bands costs 513K — the same data, spread out, for a hundred and seventy times the memory.
>
> If a workbook will not fit, look first at how far apart its rows are.

At the program's own limits — 16,384 cells on each of sixteen worksheets — the cells alone come to 128K and the worksheet indexes to another 16K, which is why those limits are reachable on an expanded machine and not on a standard one.

When memory runs out

Message	Means
`Out of memory`	There is no more banked memory. A larger workbook needs a machine with more of it.
`Resource limit reached`	Memory is not the problem: a fixed limit has been reached — 16,384 cells on a worksheet, sixteen worksheets, 128 number formats, 8,192 pieces of text. Appendix A lists them all.

An import that runs out does not fail. It stops, keeps what it has, and reports that not everything fitted.

The address space

For completeness, and because it explains why a program this small needs banked memory at all: of the 64K the processor can address, a program on the Commander X16 gets about 30K and the rest belongs to the machine.

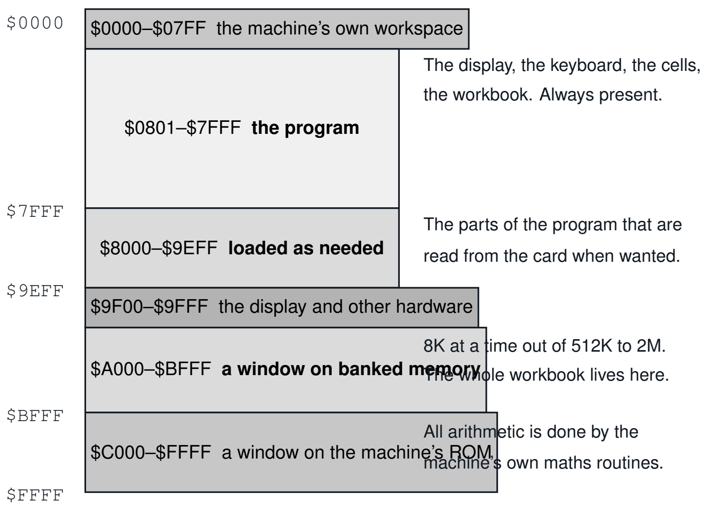


Figure D-1 The address space as Commander Calc uses it

The workbook does not live in that 30K — it lives in banked memory, reached 8K at a time through the window at $A000. The program does not fit in it either, which is why the block at $8000 is filled from the card with whichever part of the program is wanted next.

NOTE That is the reason for two things you will notice in ordinary use. Commands pause before they start, because the machine is reading from the card. And the card has to stay in the drive for the whole session, not just at startup.

If the status line says STACK!

CAUTION The program has run out of its own internal working space. It noticed rather than crashing, but nothing after that point is guaranteed.
Save the workbook **under a new name** — not over your existing file — and restart Commander Calc. Then check the saved workbook before trusting it.

APPENDIX E

Files Supplied

The program

All seventeen must be in one directory on one device.

File	What it is
`CMDRCALC.PRG`	The program. This is the one you load.
`OVL1.BIN`	The rest of the program. Commander Calc is larger than the machine's memory, so it keeps most of itself on the card and reads in whichever part it needs at the time. Which file holds what is the program's business; all you need to know is that every one of them has to be there.
`OVL2.BIN`	
… *through* …	
`OVL15.BIN`	
`OVL16.BIN`	

About 135K in total. The byte counts are from one build and move as the program is worked on; what matters is that all seventeen files are present.

Example workbooks

In Commander Calc's own format

Open these with **F3**.

INVOICE.X16S

One worksheet. A small invoice: currency, a date, a computed line total from `=B5*C5`, a **SUM** of the lines, VAT as a percentage, and a total due.

Shows what the currency, date and percentage formats look like — and, since none of them can be set from the keyboard, it is the shortest way to see them at all.

RAIN.X16S

Two worksheets, *Readings* and *Summary*. Twelve months of rainfall on the first; the second reads across the sheet boundary with `=SUM(Readings!B2:B13)`, **MAX**, **MIN**, **AVERAGE**, **COUNT** and an **IF**.

The one to look at if cross-sheet references are what you are trying to understand.

KINDS.X16S

One worksheet, one row per kind of thing a cell can be: text, a number, a negative number, a very small number, currency, a percentage, a fixed three places, a date, a boolean, `=2^10` which should read 1024, and `=1/0` which should read **`#DIV/0!`** and does.

A reference card for what everything looks like on screen.

NDQNAT.X16S

Two worksheets: five hundred days of real NASDAQ Composite prices, and a summary. Large.

This one is a stopwatch. It is exactly the same data as the `.XLSX` version, and the point is the difference in how long the two take to open — seventeen seconds against fifty-one.

Excel workbooks

Open these with F7.

BUDGET.XLSX

One worksheet. Money, percentages and totals across a ten-row table of categories, using currency and percentage formats, **SUM**, **MAX**, **AVERAGE**, subtraction and division.

STOCK.XLSX

One worksheet, deliberately in no order at all. Mixed text and number columns, line values from `=C*D`, **SUM** and **COUNT**.

Made for trying **Data > Sort up** on: put the cursor in any column, on the first row of data, and watch whole rows move. It is also the best of these for trying the **Chart** menu against.

TOTALS.XLSX

Three worksheets: *Q1*, *Q2* and *Summary*. The third reads the first two — `=Q1!B5`, `=MAX(Q1!B2:B4,Q2!B2:B4)` — so it also demonstrates that recalculation crosses worksheets.

FUNCS.XLSX

One worksheet with one row per function and the expected answer written beside it. Every one of the fourteen, plus the ^ operator, a comparison, and `=A20/0` producing **`#DIV/0!`** on purpose.

The quickest way to check that a build is working, and a usable crib sheet.

> **NOTE** Every formula in these files is one Commander Calc can compile. They are not test fixtures — they are small spreadsheets built to be used, each exercising a different part of the program.

Files the program creates

File	When
`CMDRCALC.TMP`	During every save. Deleted, or renamed over your workbook, when the save finishes. If you find one lying about, a save was interrupted — your previous workbook is intact and the temporary file can be deleted.
`CHART.BMP`	When S is pressed on a chart. Always that name, so each one overwrites the last. Chapter 14.

APPENDIX F

Differences and Known Limitations

Everything Commander Calc does not do, or does differently, in one list. Two kinds of entry are mixed here and marked apart: **absences**, which are work that has not been done, and **defects**, which are work that was done wrong.

Absences

Selection and editing

- **No fill, no fill down, no fill series.** A column of similar formulas is built by copying a block and pasting it, which does adjust the references — but there is no one-key fill.
- **No cut or move.** Copy and paste, then clear what you left.
- **No multi-level undo.** One cell, one level, and a pasted or cleared block cannot be undone at all.
- **No Find Next.** There is no need for one: repeating **Find** walks the matches. See Chapter 13.
- **Replace does not touch formulas**, and cannot be undone.
- **No Go to.** You cannot type an address to jump to it.
- **No insert or delete of a single cell.** Whole rows and whole columns only; neighbours cannot be shifted on their own.

Formatting

- **No formatting commands of any kind.** Number formats, bold, alignment and column widths are all supported by the display and both file formats — and none of them can be set from the keyboard. They arrive from imported files only.
- **One currency symbol, `$`**, and no thousands separators.
- **One date format, `YYYY-MM-DD`.**
- **No colours, borders, fonts or cell shading.**
- **No split screen, no zoom, no hidden rows or columns.** There *are* freeze panes — see Chapter 4.
- **No printing.**

Formulas

- **Fourteen functions.** No text functions, lookup functions, statistical functions beyond **AVERAGE**, financial functions or date functions. Chapter 18 has the full list of what is missing.
- **No `&` concatenation** and no `%` postfix.
- **No named ranges**, no whole-column references (`A:A`), no three-dimensional ranges across worksheets, no array formulas.
- **No R1C1 notation.**
- **No iterative calculation.** (There *is* a manual recalculation mode — see Chapter 20.)
- **Semicolon is not accepted** as an argument separator, and a tab is not treated as whitespace.

Elsewhere

- **No macros, windows, or linked workbooks.**
- **Three chart types**, all of one column, at most sixteen values: bars, a line and a pie. None is stored in the workbook, though any of them can be written out as a `.BMP`. No stacked, scatter or area charts. Chapter 14.
- **No way to change device or directory** from inside the program.
- **No way to type a date.**

Differences in behaviour

Things that exist here and work differently from the way another spreadsheet would do them.

Case	Commander Calc	Elsewhere
`=2^3^2`	64	512 in mathematics; 64 in Excel
`=-2^2`	4	−4 in mathematics; 4 in Excel
MIN or **MAX** of nothing	0	0 in Excel
AVERAGE of nothing	**`#DIV/0!`**	**`#DIV/0!`** in Excel
MOD sign	Takes the divisor's	Same in Excel; differs from C
INT of −2.7	−3	−3 in Excel; −2 if truncating
Circular reference	**`#CYCLE!`** in the cell	0 and a status warning in Excel
A number below 1	`.5`, no leading zero	`0.5`
Value too wide for a column	Cut off silently	`####`
`$` in a reference	Parsed and ignored	Fixes the reference on copy
Deleting a referenced row	Points at the replacement	**`#REF!`**

Where Commander Calc and Excel agree, that is deliberate: the point of an import format is that the numbers come out the same.

Defects

These are wrong rather than absent. They are documented here because a manual that quietly omitted them would be worse than useless.

END goes to the wrong place

END moves to column `IV` — the last column that can exist — rather than to the last column in use. In an empty worksheet that is a jump of 255 columns into nothing. SHIFT + HOME gets you back.

A sorted range is not rewritten, and cannot be

Inserting rows, deleting rows, inserting and deleting columns, sorting, undoing a sort and pasting all rewrite the references inside formulas so that they go on meaning the same cells. One case is beyond that treatment.

> **CAUTION** **A range spanning rows that a sort moves is wrong afterwards.** `=SUM(B2:B10)` names nine consecutive rows; a sort scatters those rows, and no rewriting can express where they went, because they are no longer next to each other. Commander Calc asks before sorting rows that hold formulas — `ranges over sorted rows may be wrong` — and that is the case it is asking about.
> Keep range formulas out of the rows being sorted: in a summary block above the table, or on another worksheet.

Rewriting has three further gaps, all documented in Chapter 12: a reference naming another worksheet is left alone; a reference pointing straight at a deleted row silently ends up on whatever replaced it, where Excel would show **`#REF!`**; and a formula over 128 characters long is not rewritten at all.

Text comparison is by identity, not by content

> **CAUTION** Two cells both containing `abc` compare as unequal, because each piece of text in a workbook is a separate object and comparison tests whether they are the same object. `="a"="a"` is `FALSE`.
> Worse, unequal text compares as greater in both directions: `="a">"b"` and `="b">"a"` are both `TRUE`, and < on text is always `FALSE`.
> Comparison is reliable on numbers, dates and booleans only.

The time format is not implemented

A cell formatted as a time displays its raw fractional number rather than a time. This is deliberate — visibly unfinished was preferred to plausibly wrong — but it is a gap, not a design.

A date-formatted cell holding an out-of-range value

A cell with a date format whose value is negative, or too large to be a day number, produces unpredictable rubbish rather than an error. Day number 0 displays as `1900-01-00`, which is at least recognisable as nonsense.

The note on an import result box is wrong

The counts on an import result box are correct. The parenthesised note after them is off by one, so the wrong phrase can appear — or an empty pair of brackets. **Read the counts, not the note.**

Cosmetic

- The highlight on an open menu's name is four characters wide whatever the name is, so **Sheet** shows as `Shee` with a stray `t`.
- One character of the formula bar, between the separator and the contents, is never painted and shows through in the worksheet colour.
- Mouse hit-testing inside menus and dialogue boxes assumes an 80 by 60 display, and is inaccurate in any other text mode.
- The Import file list and its busy box are both titled `Open`.
- A failure in **New** or **Save as** is reported as `Save failed`.

Limits that will surprise you

Not defects, but sharp edges worth knowing about in advance.

Limit	Consequence
32 files in a dialogue	A directory with more workbooks than that has some that cannot be opened, with no indication.
95 characters per CSV field	Longer text is truncated silently on import.
80 characters per entry	Long formulas must be built in a file and imported.
10 live values per formula	A function with eleven or more arguments gives **`#ERR11!`**.

Limit	**Consequence**
128K per XLSX part	Bigger than the cell limit suggests, and it is about the file rather than the data.
512 formulas per XLSX import	The rest keep their cached values.
4,096 rows per sort	`too many rows to sort.`
15 characters at the Find prompt	

APPENDIX G

Glossary

Term	Meaning
Address	The name of a cell: its column letter followed by its row number, as in `B9`.
Banked RAM	Memory reached 8K at a time through a window in the address space, rather than directly. The whole workbook lives here. See Appendix D.
Boolean	A cell holding `TRUE` or `FALSE`. Counts as 1 or 0 in arithmetic.
Cell	One box of the worksheet. Holds a number, text, a date, a boolean, a formula or an error value — or nothing.
Cell cursor	The reverse-video block showing which cell the keyboard is aimed at.
Clipboard	The one cell held by **Copy** for **Paste**.
Chart	A picture of the numbers in the cursor's column — bars, a line or a pie — drawn on the whole screen. Not stored in the workbook, but **S** writes it to the card as `CHART.BMP`. Chapter 14.
CMDR-DOS	The disk operating system of the Commander X16, which Commander Calc uses for all file access.
Column	A vertical line of cells, named `A` to `IV`.
Commit	To finish an entry so it is stored, with **RETURN**, **TAB**, **SHIFT**+**TAB** or an up or down arrow.
CSV	Comma-separated values: plain text, one line per row, fields separated by commas.
Current cell	The cell the cursor is on. Every one-cell command acts on it.
DEFLATE	The compression method used inside an `.XLSX` archive. Commander Calc can decompress it but cannot produce it, which is why exported `.XLSX` files are large.
Edit mode	The state in which typing goes into a cell. The status line says `EDIT`.
Error value	A value such as **`#DIV/0!`** that a formula produces when it cannot produce a number. Chapter 21 lists them.
Formula	A cell entry beginning with =, evaluated to produce the cell's value.
Freeze	Pinning the rows above the cursor and the columns to its left so that they stay on screen while the rest scrolls. Chapter 4.

Term	Meaning
Formula bar	The second line of the screen, showing the current cell's address and its actual contents.
General	The default number format: up to nine significant digits, no trailing zeros, no leading zero below 1.
Handle	The bank-and-offset pair by which everything in banked RAM is addressed, in place of a pointer.
HostFS	The emulator's way of presenting a directory on the host computer as though it were a disk on unit 8.
ISO mode	The character mode Commander Calc puts the machine into at startup, which makes the keyboard produce ordinary ASCII.
KERNAL	The Commander X16's built-in operating system.
MBF	Microsoft Binary Format: the five-byte floating-point representation Commander Calc stores numbers in, and the one the machine's ROM arithmetic uses. Appendix C.
Number format	How a cell's value is displayed — General, Integer, Decimal, Currency, Percent, Date, Time or Text. Chapter 19.
PETSCII	The Commodore character encoding. Commander Calc works in ASCII internally and converts only where CMDR-DOS requires it, in file names.
Range	A rectangle of cells, written `A1:B9`. Only **SUM**, **AVERAGE**, **MIN**, **MAX** and **COUNT** accept one.
Ready mode	The state in which keys move the cursor and run commands. The status line says `READY` or `MODIF`.
Recalculation	Working out every formula's value again. It happens after every change, over every worksheet, unless automatic recalculation has been switched off. Chapter 20.
Reference	The name of a cell or a range inside a formula.
Row	A horizontal line of cells, numbered 1 to 65,535.
Serial	The number that stands for a date: 1 is 1 January 1900.
Status line	The second line from the bottom, showing the mode, the current cell and the free memory.
Style	A stored combination of number format, decimal places, bold and alignment. A cell names one by number. There are at most 128 in a workbook.
Tab	The name of a worksheet, as shown on the tab line near the bottom of the screen.
Text pool	The store holding every piece of text in the workbook. Limited to 8,192 entries, and it does not share identical text.

Term	Meaning
Used range	The rectangle from `A1` to the last row and column of a worksheet with anything in them. What **Export** writes and what **Sort** covers.
VERA	The Commander X16's video chip. Commander Calc writes to it directly rather than going through the KERNAL, which is why the display is fast and why `-echo` cannot capture it.
Workbook	Everything in memory: up to sixteen worksheets, saved as one `.X16S` file.
Worksheet	One grid of cells, with its own name and its own tab.
X16S	Commander Calc's own file format. Appendix B.
XLSX	The Excel workbook format: a ZIP archive of XML documents. Commander Calc reads and writes it. Chapter 9.

Index